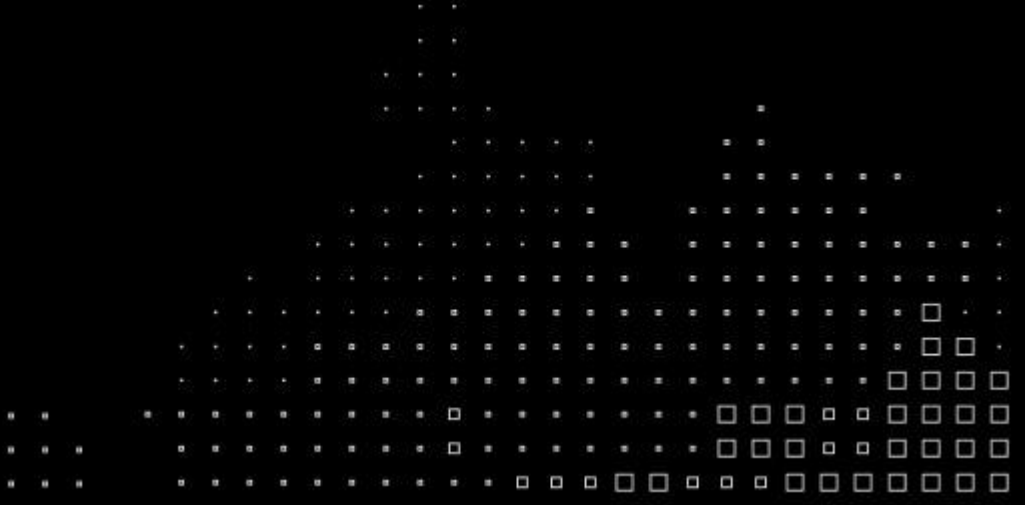
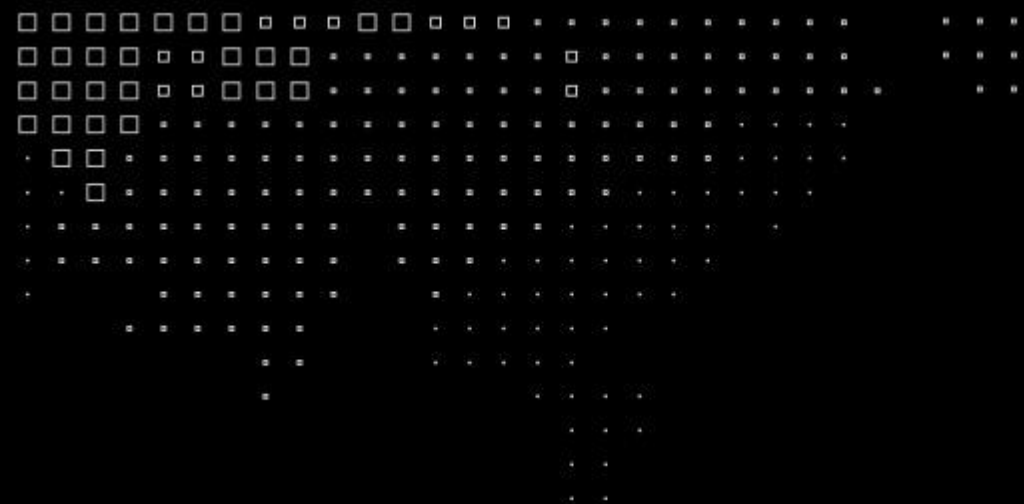


MBA⁺

**AI Engineering &
Multi Agents**



**MBA⁺****Data Architecture,
Integration and Ingestion**

Prof.: Leandro M. Mendes
Email: profleandro.mendes@fiap.com.br

Material de Referência:
Prof.: Ivan Gancev
Email: profivan.gancev@fiap.com.br



Recapitulando...

- Definições de dado, arquitetura de dados, bancos de dados e tipos de dados
- Data warehouse (surgimento, aplicações e características)
- Data Lake (Hadoop, processamento distribuído, arquiteturas e ingestão)
- Data Lakehouse (combinação das melhores características anteriores)
- Data Mesh (domínios de dados, dado como produto e proximidade com consumo)

Plano do curso

Data Architecture, Integration and Ingestion

- **Aula 1 — Ambientes modernos de dados** Tipos de dados, SGBDs, arquiteturas de referência: Data Warehouse, Data Lake, Data Lakehouse e Data Mesh.
- **Aula 2 — Modelos de banco de dados I** Bancos relacionais e colunares: modelagem, escalabilidade.
- **Aula 3 — Modelos de banco de dados II** Bancos de documentos, grafos e vetores.
- **Aula 4 — Governança, integração e ingestão** Qualidade e catalogação de dados, pipelines batch e real-time, ferramentas de ingestão.
- **Aula 5 — APIs, visualização e CDC** Arquiteturas orientadas a eventos, APIs e visualização de dados.

Plano de aula

Revisão / finalização da aula anterior

1. Bancos de dados relacionais

1. Fundamentos de bancos relacionais e SQL
2. Laboratório com SQL – parte 1 (modelo)
3. Laboratório com SQL – parte 2 (setup)
4. Laboratório com SQL – parte 3 (implementação)

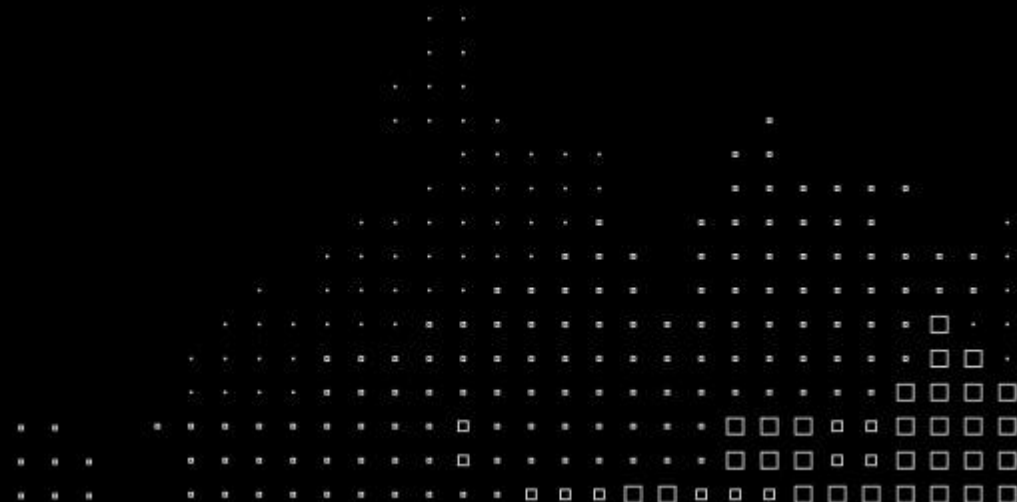
2. Bancos de dados NoSQL

1. Fundamentos de bancos de dados NoSQL
2. Bancos de dados colunares
3. Prática com Cassandra

Data Architecture, Integration and Ingestion

Capítulo 1: Bancos de dados relacionais

Fundamentos de bancos relacionais e SQL



O problema que o modelo relacional veio resolver

Antes do modelo relacional, dados eram armazenados em arquivos sequenciais ou hierárquicos — sem padronização, sem integridade, sem linguagem comum. Cada aplicação definia sua própria estrutura.

Em 1970, Edgar F. Codd propôs o modelo relacional: dados organizados em tabelas com estrutura predefinida, relacionamentos formais entre entidades e uma linguagem declarativa para consulta e manipulação.

O modelo resolve três problemas centrais:

Integridade

Regras que garantem que os dados sejam válidos e consistentes

Independência

A aplicação não precisa saber como os dados estão fisicamente armazenados

Padronização

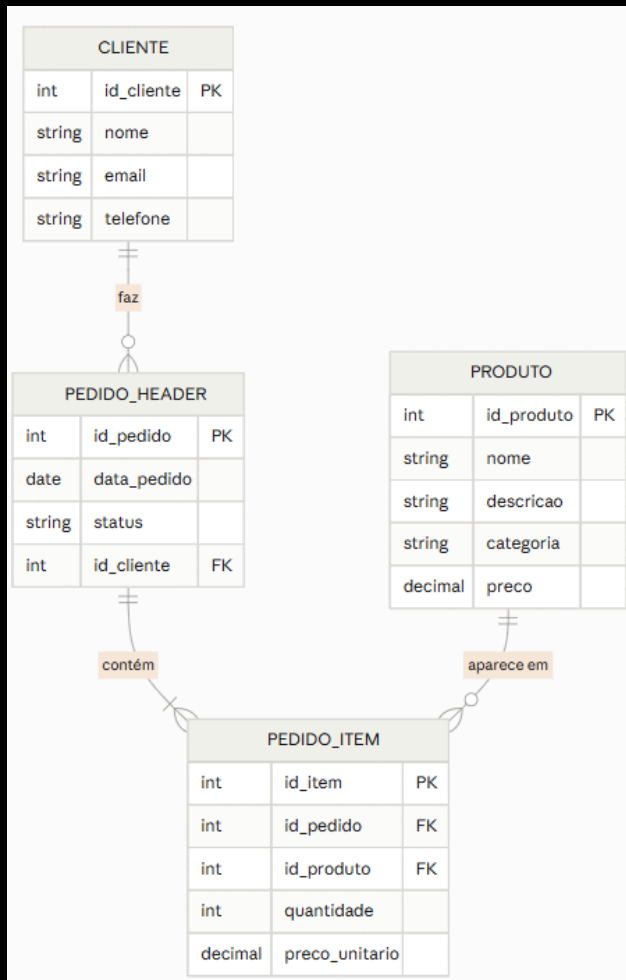
Qualquer sistema que implemente o modelo fala a mesma linguagem

Características do modelo relacional

- **Modelo estruturado:** esquema definido antes da inserção dos dados (schema-on-write). Cada tabela tem colunas com tipos fixos. Dado fora do esquema não entra.
- **Chaves primária (PK) e estrangeira (FK):** É o mecanismo que estabelece relacionamentos e garante unicidade e integridade referencial.
- **Propriedades ACID:** garantem que as operações no banco sejam confiáveis mesmo em falhas ou acessos concorrentes. Serão detalhadas no próximo bloco.
- **SQL:** linguagem padronizada (ISO/ANSI) para definição, manipulação, controle e transação de dados. Implementada por todos os principais SGBDs relacionais.

Referências: Codd, E.F. (1970). A Relational Model of Data for Large Shared Data Banks. Communications of the ACM, 13(6).
Elmasri, R.; Navathe, S.B. (2016). Fundamentals of Database Systems. 7ª ed. Pearson.

O modelo relacional



O diagrama ao lado ilustra um modelo relacional simples com os elementos descritos logo abaixo:

- **Entidade**: objeto do mundo real sobre o qual se quer armazenar dados (ex: Produto, Cliente)
- **Atributo**: propriedade de uma entidade (ex: nome, preço, data de cadastro)
- **Chave primária (PK)**: atributo ou conjunto de atributos que identifica unicamente cada registro
- **Chave estrangeira (FK)**: atributo que referencia a PK de outra tabela, estabelecendo o relacionamento
- **Cardinalidade**: define a natureza do relacionamento: um-para-um, um-para-muitos, muitos-para-muitos

Structured Query Language (SQL)

SQL foi padronizado pela ISO/ANSI e é implementado com variações por todos os principais SGBDs relacionais. Abaixo os principais componentes da linguagem:

DDL: Data Definition Language: define e modifica a estrutura do banco

→ CREATE, ALTER, TRUNCATE, DROP, DESCRIBE, RENAME

DML: Data Manipulation Language: opera sobre os dados

→ INSERT, UPDATE, DELETE, SELECT

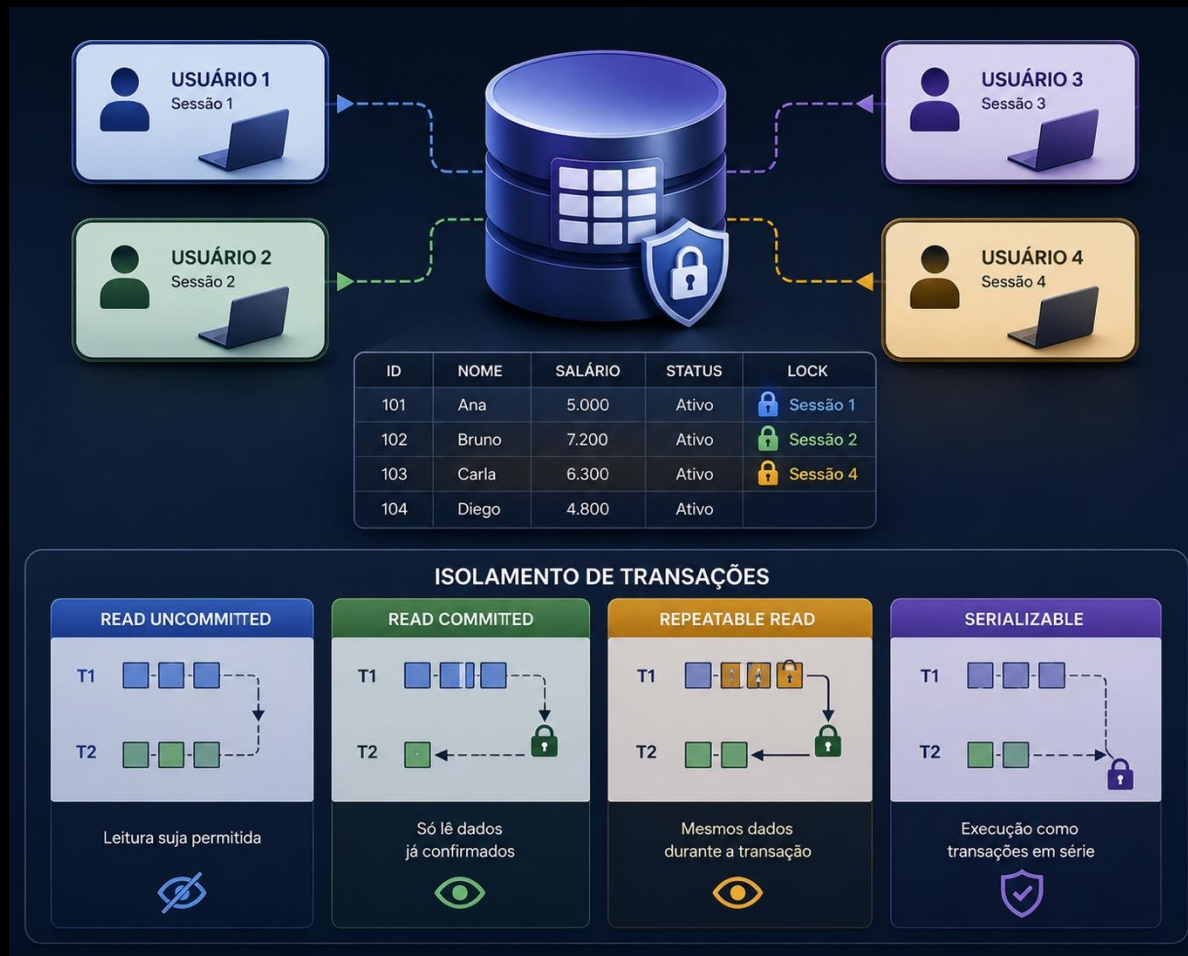
DCL: Data Control Language: controla acesso e permissões

→ GRANT, REVOKE

TCL: Transaction Control Language: controla acesso e permissões

→ ROLLBACK, COMMIT

Controle de sessões



Quando múltiplos usuários acessam o mesmo dado simultaneamente, o SGBD precisa garantir que operações concorrentes não corrompam o estado do banco. Esse controle é feito com mecanismos como:

- **Lock:** o SGBD bloqueia um registro ou tabela durante uma operação de escrita, impedindo que outra sessão altere o mesmo dado ao mesmo tempo
- **Isolamento de transações:** define o quanto uma transação em andamento é visível para outras sessões. Configurável por nível: *read uncommitted*, *read committed*, *repeatable read*, *serializable*
- **Deadlock:** situação em que duas sessões se bloqueiam mutuamente esperando recursos uma da outra. O SGBD detecta e aborta uma das transações automaticamente

Metadado

Metadados são os dados que descrevem a estrutura do banco — o "dado sobre o dado".

Todo SGBD mantém um conjunto de tabelas (catálogo ou dicionário de dados) que armazenam:

- Estrutura de cada tabela: colunas, tipos, constraints
- Índices criados e suas estatísticas de uso
- Usuários, permissões e roles
- Sessões ativas e queries em execução
- Parâmetros de configuração do servidor

O otimizador de queries usa os metadados para decidir o plano de execução mais eficiente — índice ou full scan, ordem de joins, paralelismo.

Essas tabelas são consultáveis via SQL.
No MySQL: INFORMATION_SCHEMA.

No PostgreSQL: pg_catalog.
No Oracle: ALL_TABLES, DBA_OBJECTS.

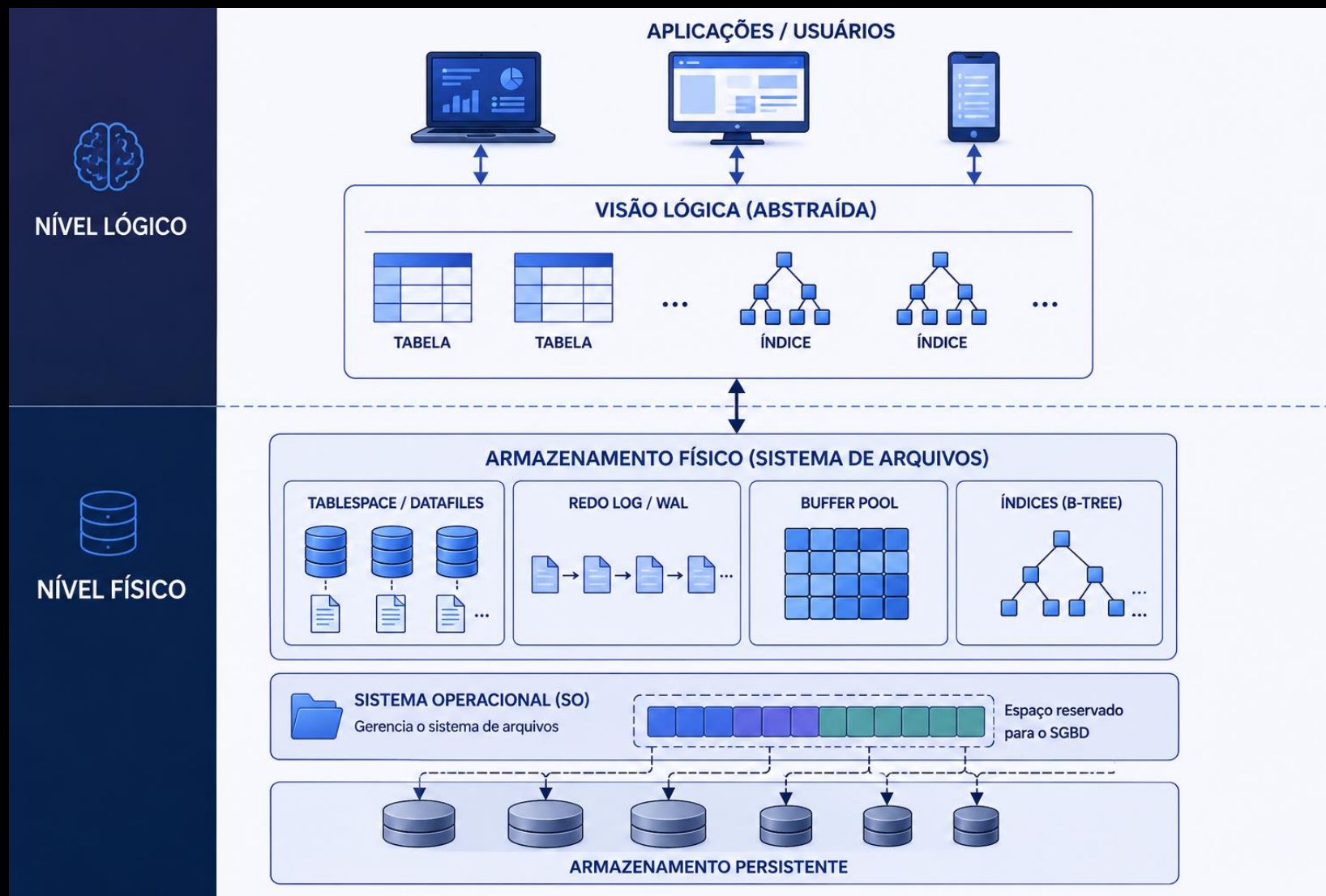
Armazenamento

O SGBD abstrai o armazenamento físico da aplicação, mas internamente organiza os dados em estruturas específicas no sistema de arquivos:

- Tablespace / datafiles — arquivos físicos onde as tabelas e índices são armazenados. O DBA controla localização, tamanho e crescimento
- Redo log / WAL (Write-Ahead Log) — registro sequencial de todas as operações antes de serem aplicadas aos dados. Garante durabilidade e permite recovery em caso de falha
- Buffer pool — área de memória onde o SGBD mantém páginas de dados frequentemente acessadas, reduzindo I/O em disco
- Índices — estruturas auxiliares (tipicamente B-tree) que aceleram consultas ao custo de espaço em disco e overhead em escritas

Essa separação entre dado lógico e armazenamento físico é um dos princípios centrais do modelo relacional — a aplicação opera sobre tabelas, não sobre arquivos.

Armazenamento



Note que o SGBD “negocia” com o Sistema Operacional e reserva o espaço necessário, de modo que ele se torna temporariamente dono desse espaço, para organizá-lo da forma mais otimizada quanto possível.

Procedimentos (*Stored Procedures*)

Stored procedures são rotinas armazenadas e executadas diretamente no SGBD, escritas em linguagens procedurais proprietárias (PL/SQL no Oracle, T-SQL no SQL Server, PL/pgSQL no PostgreSQL).

- Encapsulamento de lógica de negócio — regras complexas ficam no banco, não na aplicação. Reduz tráfego de rede e garante consistência independente do cliente que acessa
- Reutilização — uma procedure pode ser chamada por múltiplas aplicações ou outros objetos do banco
- Controle de acesso granular — é possível conceder acesso à procedure sem expor as tabelas subjacentes
- Usos típicos — processos batch, importações, integrações, cálculos complexos sobre grandes volumes de dados

Trade-off relevante: lógica no banco dificulta versionamento, testes automatizados e portabilidade entre SGBDs.

Gatilhos (*Triggers*)

Triggers são rotinas disparadas automaticamente pelo SGBD em resposta a eventos de manipulação de dados (INSERT, UPDATE ou DELETE) em uma tabela específica.

- BEFORE trigger — executa antes da operação. Útil para validação ou transformação do dado antes de persistir
- AFTER trigger — executa após a operação. Útil para auditoria, propagação de mudanças ou atualização de tabelas derivadas
- Granularidade — podem ser configuradas por linha (FOR EACH ROW) ou por statement
- Usos típicos: auditoria de alterações, atualização automática de campos de controle (created_at, updated_by), propagação de eventos para outras tabelas.
- Trade-off: triggers executam de forma implícita — quem olha o DML não sabe que há lógica adicional sendo disparada. Em sistemas complexos, isso aumenta o custo de manutenção e depuração.

Transações

Uma transação é uma unidade lógica de trabalho composta por uma ou mais operações que devem ser executadas de forma atômica — tudo ou nada.

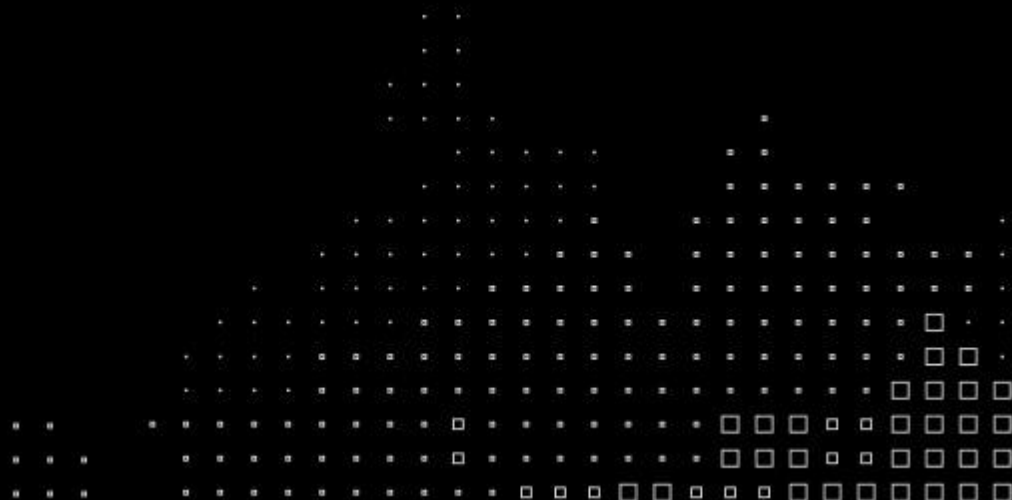
As propriedades que garantem o comportamento correto das transações são definidas pelo modelo ACID:

- Atomicidade — todas as operações da transação são efetivadas juntas ou nenhuma é. Em caso de falha, o banco volta ao estado anterior (rollback)
- Consistência — a transação leva o banco de um estado válido a outro estado válido. Constraints, regras e integridade referencial são sempre respeitadas
- Isolamento — transações concorrentes não interferem entre si. O resultado é equivalente à execução sequencial
- Durabilidade — uma vez confirmada (commit), a transação persiste mesmo em caso de falha de hardware ou energia

Data Architecture, Integration and Ingestion

Capítulo 1: Bancos de dados relacionais

Laboratório com
SQL – parte 1
(modelo)



O que vamos fazer

Nos próximos exercícios vamos construir e operar um modelo relacional completo — modelagem, criação de tabelas, inserção de dados e consultas analíticas. Como vamos trabalhar:

- 1. Entender o problema e desenhar a solução:** A partir de um cenário de negócio real, vamos identificar as entidades, seus atributos e relacionamentos, e produzir um modelo de dados que responda às perguntas do negócio.
- 2. Implementar o modelo com apoio de IA:** Com o modelo desenhado, vamos usar IA generativa para acelerar a geração dos scripts de criação, população e consulta do banco — e executá-los no ambiente de sua escolha.

O laboratório pode ser executado em duas opções, escolha a que melhor se adapta ao seu setup:

- **Opção A — SQLite no Google Colab:** sem instalação, sem conta. Execução incremental diretamente no notebook
- **Opção B — Docker + MySQL:** ambiente mais próximo de produção. Requer Docker instalado localmente

Contexto para o laboratório

Cenário de negócio

Uma rede de varejo opera um armazém central com múltiplos departamentos. Toda a operação de estoque precisa ser registrada em um modelo relacional que garanta integridade entre os dados de inventário e as movimentações.

Estrutura base — fornecida pelo professor

- `invent_mestre` — catálogo central de todos os itens do armazém
- `transac_mestre` — registro de todas as movimentações de estoque

Sua tarefa

Escolher um departamento. Desenhar o modelo de dados necessário. Gerar os scripts para criação da base de dados (DDL) e para inserir um primeiro conjunto de dados (DML). As tabelas do seu departamento devem estar conectadas à estrutura base via chave estrangeira. Desenvolver consultas para responder perguntas analíticas sobre os dados. Nota: vamos usar IA para nos apoiar.

Laboratório Parte 1 – em grupos

Abaixo o modelo central que deve ser usado como base para criação dos modelos dos departamentos.

DEPARTAMENTO		
int	id_departamento	PK
varchar	nome	
varchar	responsavel	

classifica

INVENT_MESTRE		
int	id_item	PK
varchar	codigo_sku	
varchar	descricao	
int	id_departamento	FK
varchar	unidade_medida	
decimal	preco_custo	

movimenta

TRANSAC_MESTRE		
int	id_transacao	PK
int	id_item	FK
date	data_transacao	
varchar	tipo	
int	quantidade	
varchar	origem_destino	

sql

```
CREATE TABLE departamento (  
    id_departamento INTEGER PRIMARY KEY,  
    nome VARCHAR(100) NOT NULL,  
    responsavel VARCHAR(100)  
);  
  
CREATE TABLE invent_mestre (  
    id_item INTEGER PRIMARY KEY,  
    codigo_sku VARCHAR(50) NOT NULL UNIQUE,  
    descricao VARCHAR(200) NOT NULL,  
    id_departamento INTEGER NOT NULL,  
    unidade_medida VARCHAR(20) NOT NULL,  
    preco_custo DECIMAL(10,2) NOT NULL,  
    FOREIGN KEY (id_departamento) REFERENCES departamento(id_departamento)  
);  
  
CREATE TABLE transac_mestre (  
    id_transacao INTEGER PRIMARY KEY,  
    id_item INTEGER NOT NULL,  
    data_transacao DATE NOT NULL,  
    tipo VARCHAR(10) NOT NULL CHECK (tipo IN ('ENTRADA', 'SAIDA', 'AJUSTE')),  
    quantidade INTEGER NOT NULL,  
    origem_destino VARCHAR(200),  
    FOREIGN KEY (id_item) REFERENCES invent_mestre(id_item)  
);
```

Laboratório Parte 1 – em grupos

Com o cenário em mãos, vamos usar IA generativa para produzir o modelo do seu departamento. O papel do arquiteto não muda — a IA acelera a execução, não substitui o raciocínio.

O que pedir à IA: Use um prompt como este, adaptando para o seu departamento:

"Preciso modelar as tabelas de um departamento de [seu departamento] para um sistema de armazém. A tabela `invent_mestre` já existe com os campos `id_item` (PK), `codigo_sku`, `descricao`, `id_departamento`, `unidade_medida` e `preco_custo`. Crie uma tabela específica para [seu departamento] com FK para `invent_mestre`, incluindo os atributos relevantes para esse tipo de produto. Use `VARCHAR` com tamanho definido e `INTEGER` para chaves. "

O que avaliar no resultado

- A FK está corretamente referenciando `invent_mestre.id_item`?
- Os atributos fazem sentido para o tipo de produto?
- Os tipos de dados são adequados?
- Há atributos importantes que a IA deixou de fora?

Passo final: peça à IA para gerar os comandos de INSERT para pelo menos 10 registros, incluindo simulação de movimentações de entrada e saída. Importante que os números atuais estejam consistentes com essas movimentações.

Laboratório Parte 1 - Resultados

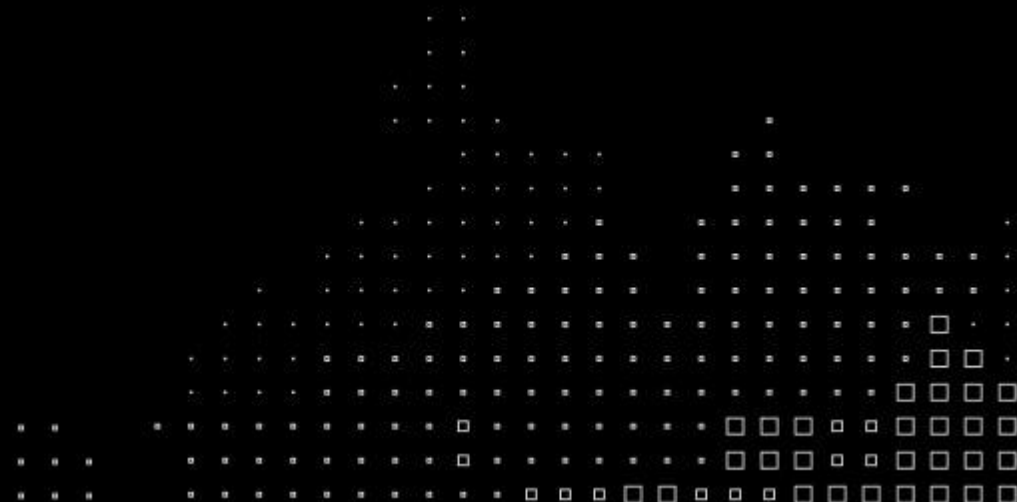
Compartilhem os resultados obtidos

- Estão satisfeitos com o modelo?
- Há algo que chamou a atenção?
- Mudariam algo no modelo mestre?

Data Architecture, Integration and Ingestion

Capítulo 1: Bancos de dados relacionais

Laboratório com
SQL – parte 2
(setup)



Opção A: SQLite no Google Colab

1. Pré-requisito: conta Google. Acesse: <https://colab.research.google.com/>

The screenshot displays the Google Colab web interface. At the top, the header includes the Colab logo and navigation links: Arquivo, Editar, Ver, Inserir, Ambiente de execução, Ferramentas, and Ajuda. Below this is a search bar and a menu with options like 'Comandos', '+ Código', '+ Texto', 'Executar tudo', and 'Copiar para o Drive'. On the left, a sidebar shows a list of notebooks under the heading 'Índice', including 'Conheça o Colab', 'Plano Pro sem custo', and various topics like 'Ciência de dados' and 'Machine learning'. A central modal window is open, titled 'Conheça o Colab', with a sub-menu showing options: 'Novo notebook no Drive', 'Abrir notebook' (with a 'Ctrl+O' shortcut), and 'Fazer upload de notebook'. A 'Rename' option is also visible. Overlaid on the right side of the interface is a large white box with a grey border containing the text: 'É necessário fazer login no Google' followed by 'É necessário fazer login com uma Conta do Google para continuar.' and a blue button labeled 'Fazer login'.

Conheça o Colab

Arquivo Editar Ver Inserir Ambiente de execução Ferramentas Ajuda

Q Comandos + Código + Texto ▶ Executar tudo Copiar para o Drive

Índice

Conheça o Colab

O Google Colab está disponível para todos.

Plano Pro sem custo

Acesse os modelos de

Conheça a API Gemini

Vamos começar

Ciência de dados

Machine learning

Mais recursos

Exemplos em destaque

+ Seção

Conheça o Colab

Arquivo Editar Ver Inserir Ambiente de execução Ferramentas

Q Comandos

Novo notebook no Drive

Abrir notebook Ctrl+O

Fazer upload de notebook

Rename

É necessário fazer login no Google

É necessário fazer login com uma Conta do Google para continuar.

Fazer login

Google Colab para instalar.

- Acesse o seletor de kernel criando ou abrindo qualquer arquivo de notebook (.ipynb) no seu espaço de trabalho, clicando em uma célula ou clicando no botão **Select Kernel** (Selecionar kernel) no canto superior direito.
- Clique em **Colab** e selecione o ambiente de execução desejado. Faça login com sua Conta do Google e pronto!

Confira mais detalhes no nosso [blog de avisos](#).

Opção A: SQLite no Google Colab

2. No novo notebook, cole os seguintes comandos:

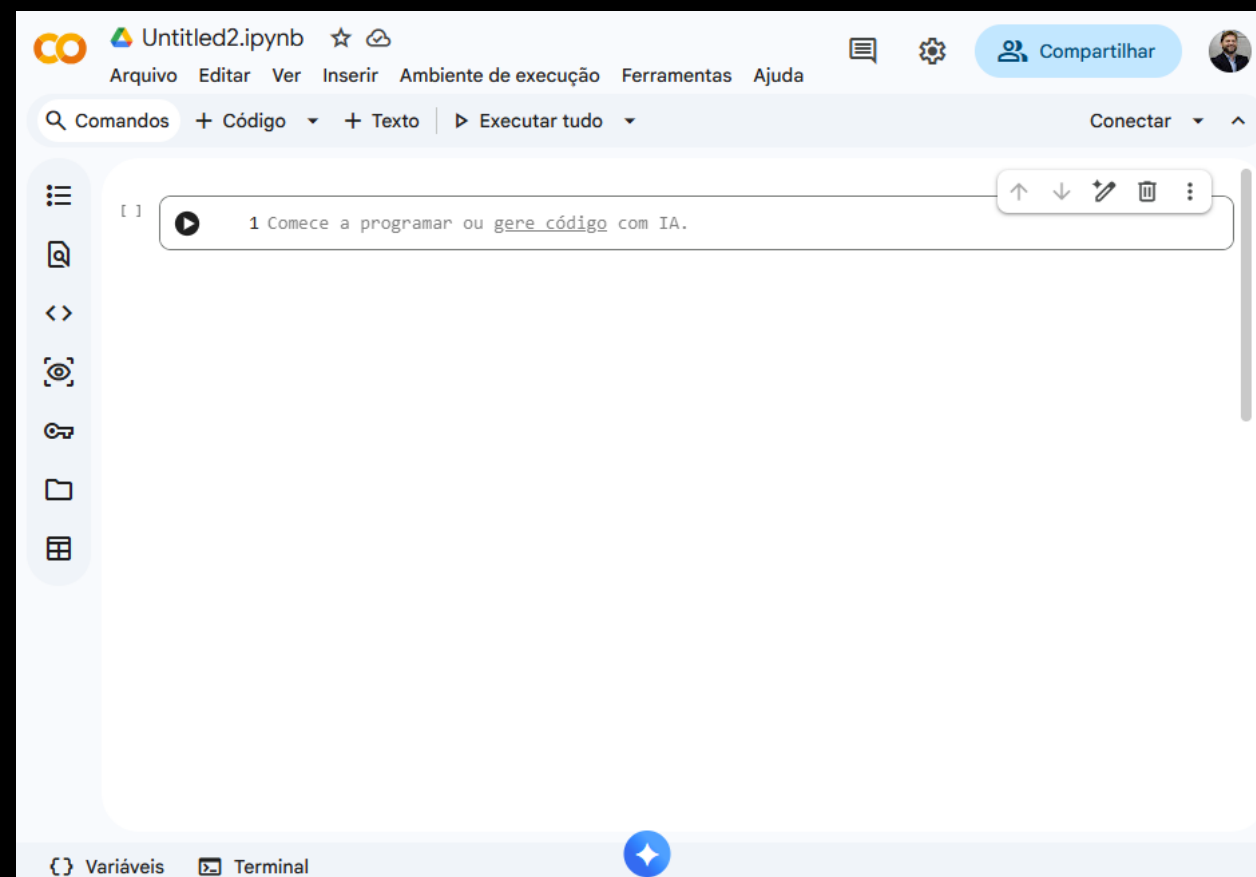
Primeiro para criar uma base em memória:

```
import sqlite3
import pandas as pd

conn = sqlite3.connect(':memory:')
```

Depois, para habilitar o uso de chaves estrangeiras:

```
conn.execute("PRAGMA foreign_keys = ON")
```



Opção A: SQLite no Google Colab

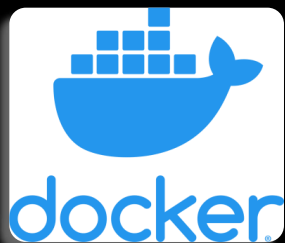
3. Para implementar o banco de dados, use:

```
conn.executescript("""  
    -- cole aqui o script gerado pela IA  
""")
```

4. E para executar consultas, use:

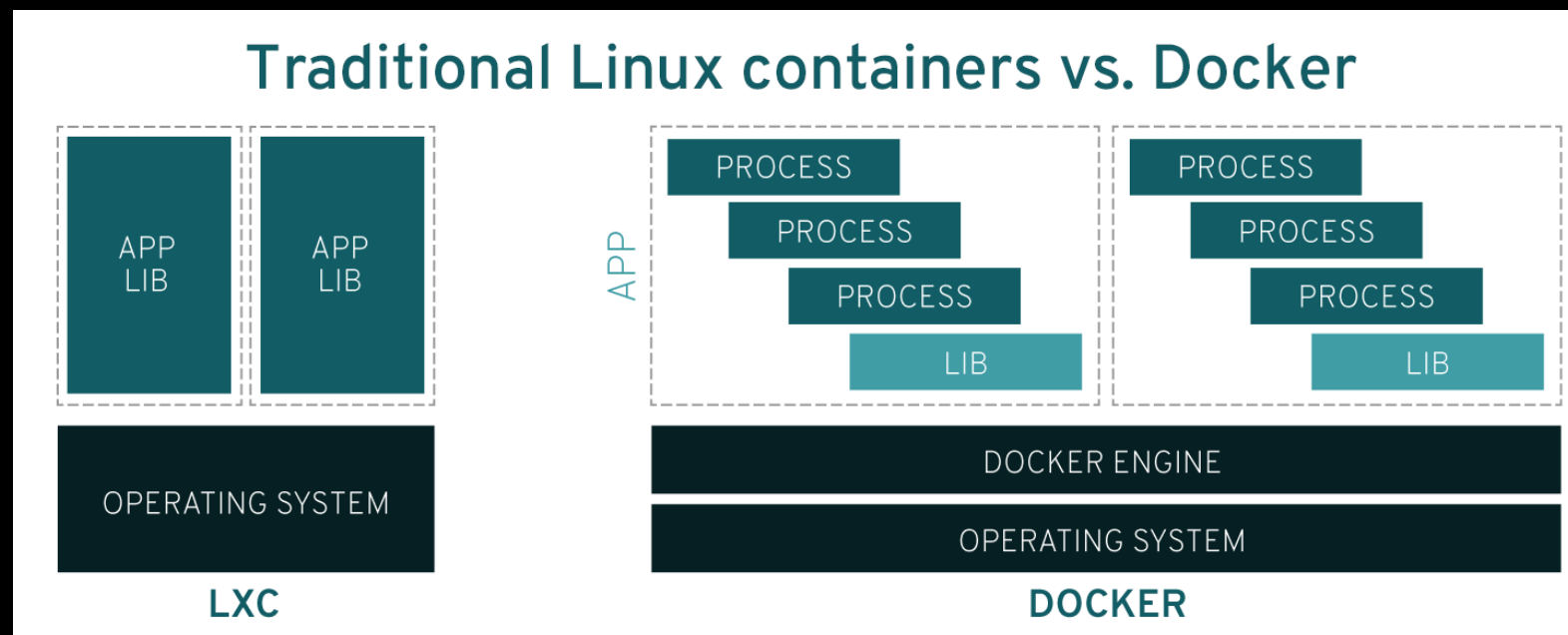
```
pd.read_sql("SELECT * FROM invent_mestre", conn)
```

Opção B: Docker + MySQL



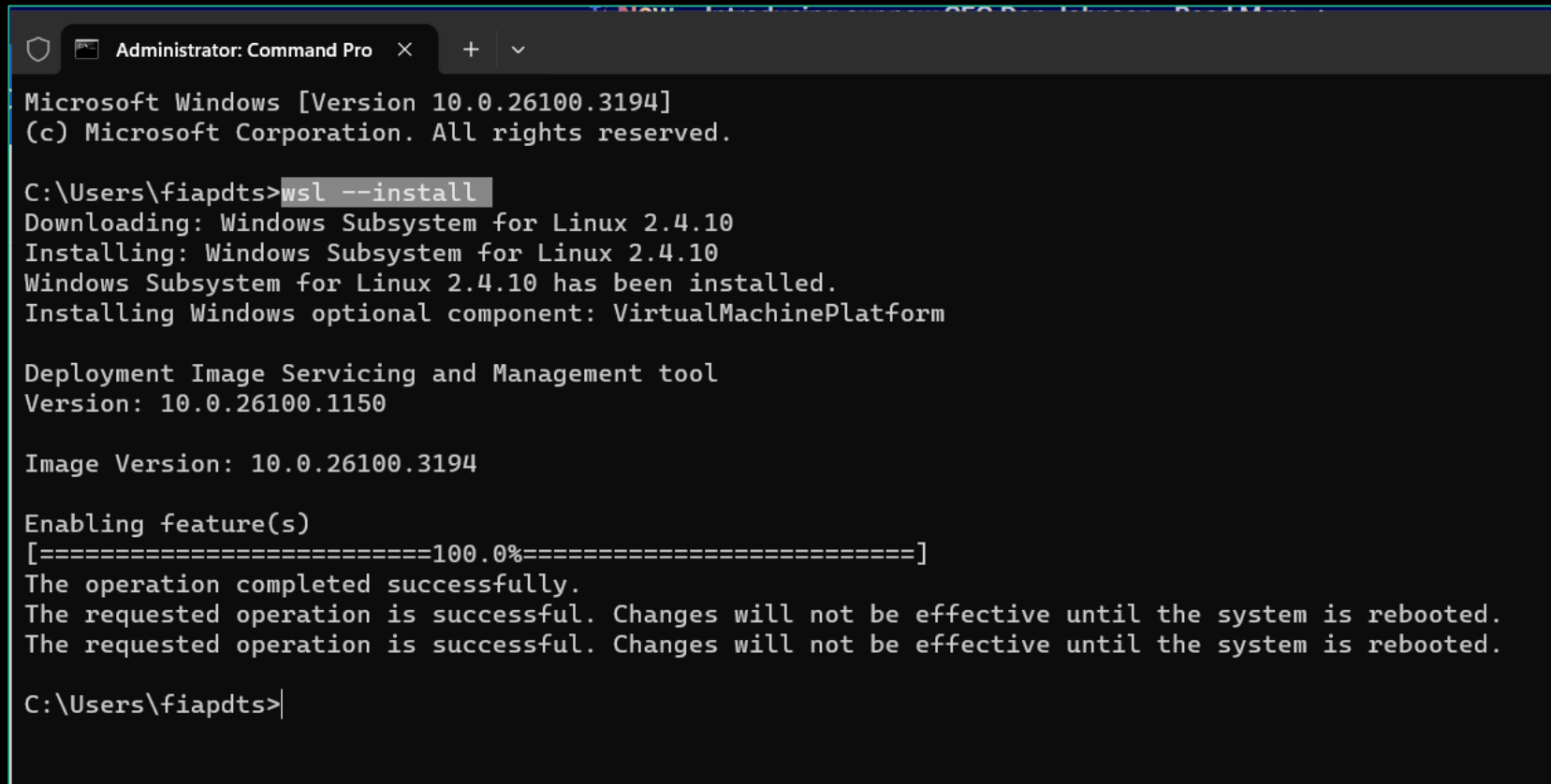
Docker é um sistema de código aberto capaz de compartilhar recursos de um mesmo sistema operacional, contudo manter um isolamento de dependências de execução das aplicações

Uma **Imagem** é um conjunto de ferramentas, bibliotecas e aplicativos que uma vez instanciados em memória passam a ter o nome **Container**.



Opção B: Docker + MySQL

1. Instalar WSL (pré-req para instalar o Docker)



```
Administrator: Command Pro  x  +  v
Microsoft Windows [Version 10.0.26100.3194]
(c) Microsoft Corporation. All rights reserved.

C:\Users\fiapdts>wsl --install
Downloading: Windows Subsystem for Linux 2.4.10
Installing: Windows Subsystem for Linux 2.4.10
Windows Subsystem for Linux 2.4.10 has been installed.
Installing Windows optional component: VirtualMachinePlatform

Deployment Image Servicing and Management tool
Version: 10.0.26100.1150

Image Version: 10.0.26100.3194

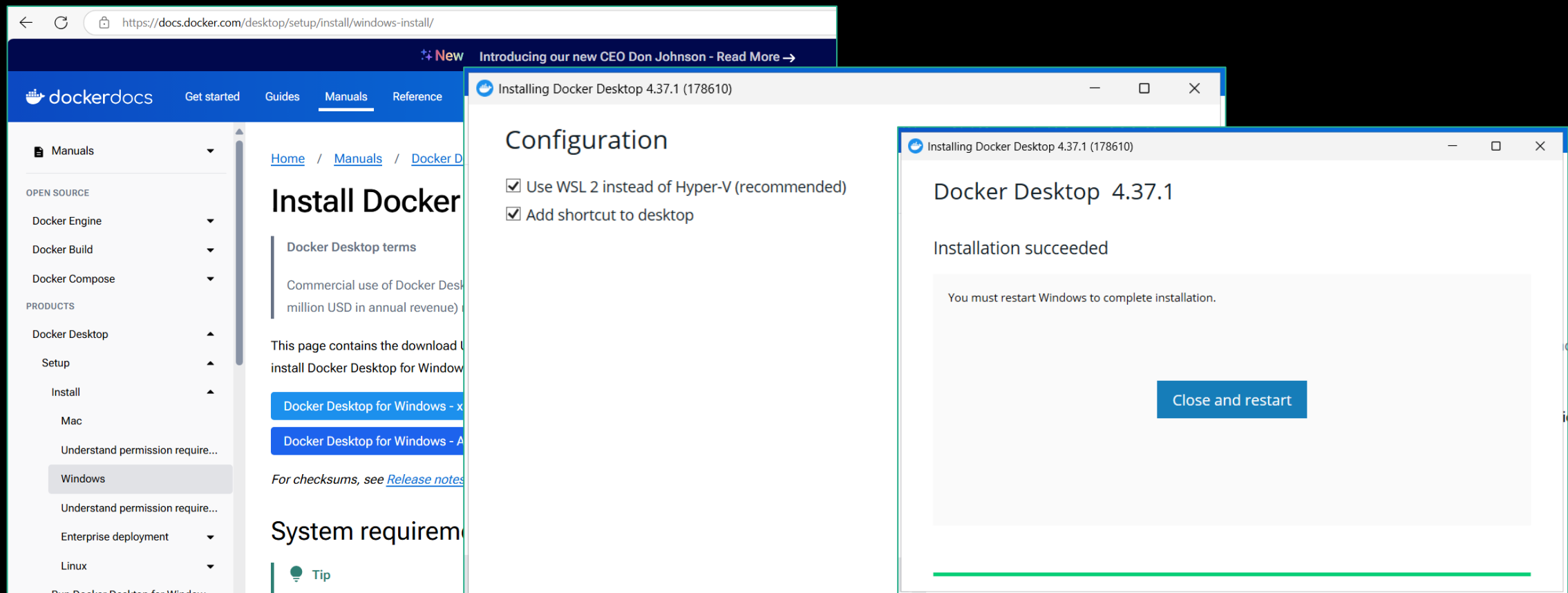
Enabling feature(s)
[=====100.0%=====]
The operation completed successfully.
The requested operation is successful. Changes will not be effective until the system is rebooted.
The requested operation is successful. Changes will not be effective until the system is rebooted.

C:\Users\fiapdts>
```

Opção B: Docker + MySQL

2. Instalar Docker (busca no Edge: “install docker desktop windows 11”)

<https://www.docker.com/products/docker-desktop/>



Opção B: Docker + MySQL

3. Subir uma imagem do MySQL executando os passos indicados ao lado para criar, iniciar e acessar o ambiente.
4. Seguir os próximos passos de configuração e execução (que iremos ver na parte 3)

Subindo o container MySQL

```
bash
docker pull mysql:8

docker run --name mysql-aula \
  -e MYSQL_ROOT_PASSWORD=fiap1234 \
  -p 3306:3306 -d mysql:8
```

Acessando o CLI

```
bash
docker exec -it mysql-aula mysql -u root -p
```

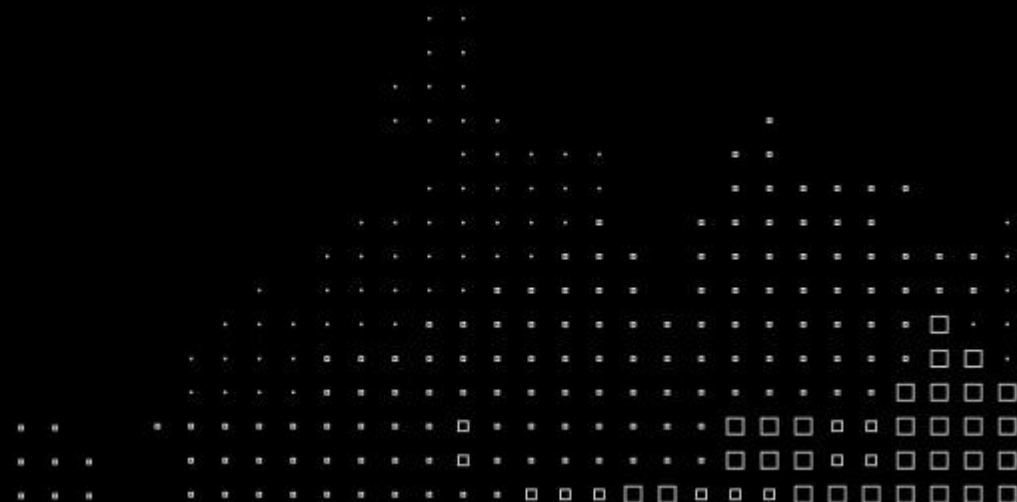
Habilitando FK no MySQL

```
sql
SET foreign_key_checks = 1;
```

Data Architecture, Integration and Ingestion

Capítulo 1: Bancos de dados relacionais

Laboratório com
SQL – parte 3
(implementação)



Roteiro de execução - individual

1. Execute o script base do professor — cria departamento, invent_mestre e transac_mestre
2. Execute o script do seu departamento gerado pela IA — cria a tabela específica com FK
3. Execute os INSERTs gerados pela IA — popula as tabelas com dados de exemplo
4. Verifique a integridade — tente inserir um item com id_departamento inexistente e observe o erro
5. Execute os SELECTs com JOIN entre sua tabela e invent_mestre
6. Responda as perguntas analíticas do próximo slide

Perguntas analíticas

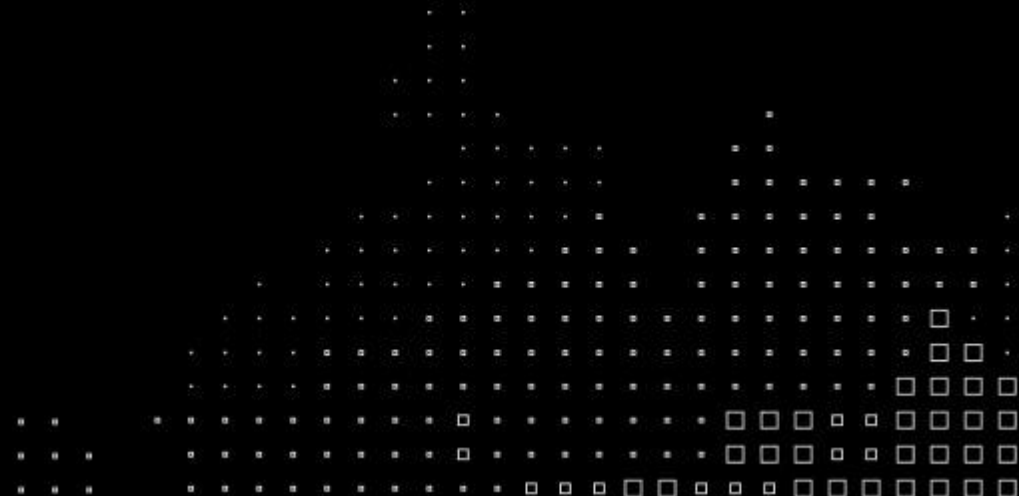
Com os dados inseridos, escreva os SELECTs que respondam às perguntas abaixo. Use IA para ajudar na construção das queries se necessário, mas valide o resultado.

1. Liste todos os itens do seu departamento com descrição, SKU e preço de custo, ordenados por preço decrescente.
2. Quantas movimentações de cada tipo (ENTRADA, SAIDA, AJUSTE) foram registradas na transac_mestre para os itens do seu departamento?
3. Qual item do seu departamento teve o maior volume total movimentado (soma de quantidade em todas as transações)?
4. Liste os itens que tiveram pelo menos uma movimentação do tipo SAIDA nos últimos 30 dias, com a quantidade total saída por item.
5. Faça um JOIN entre sua tabela de departamento e a invent_mestre mostrando os atributos específicos do produto junto com SKU e preço de custo.

Data Architecture, Integration and Ingestion

Capítulo 2: Bancos de dados NoSQL

Fundamentos de bancos de dados NoSQL



Surgimento de bancos NoSQL

No início dos anos 2000, empresas como Google, Amazon e Facebook se depararam com um problema que o modelo relacional não resolvia bem: volumes de dados na casa de petabytes, milhões de acessos simultâneos e necessidade de distribuição geográfica global.

A resposta foi desenvolver internamente sistemas de armazenamento projetados para escala horizontal — distribuídos por natureza, com consistência flexível e esquema adaptável. Esses sistemas inspiraram ou deram origem direta às tecnologias NoSQL que conhecemos hoje. Marcos relevantes:

- 2003 — Google publica o paper do GFS (Google File System)

- 2004 — Google desenvolve internamente o Bigtable

- 2006 — Google publica o paper do Bigtable — referência fundacional dos bancos colunares

- 2007 — Amazon publica o paper do Dynamo — referência fundacional dos bancos chave-valor

- 2009 — o termo NoSQL ganha tração na comunidade técnica

Referência: Chang, F. et al. (2006). *Bigtable: A Distributed Storage System for Structured Data*. OSDI.

Relembrando...

Bancos de dados NoSQL são projetados para serem altamente escaláveis e para suportar uma variedade de modelos de dados que não se encaixam bem no modelo relacional. Dois dos maiores benefícios são:

- **Escalabilidade Horizontal:** Diferente dos bancos de dados relacionais que geralmente escalam verticalmente (adicionando mais recursos a um único servidor), os bancos de dados NoSQL escalam horizontalmente, distribuindo a carga de trabalho entre vários servidores.
- **Flexibilidade de Esquema:** Os bancos de dados NoSQL permitem a inserção de dados sem a necessidade de um esquema predefinido, facilitando a adaptação a mudanças nos requisitos de dados.

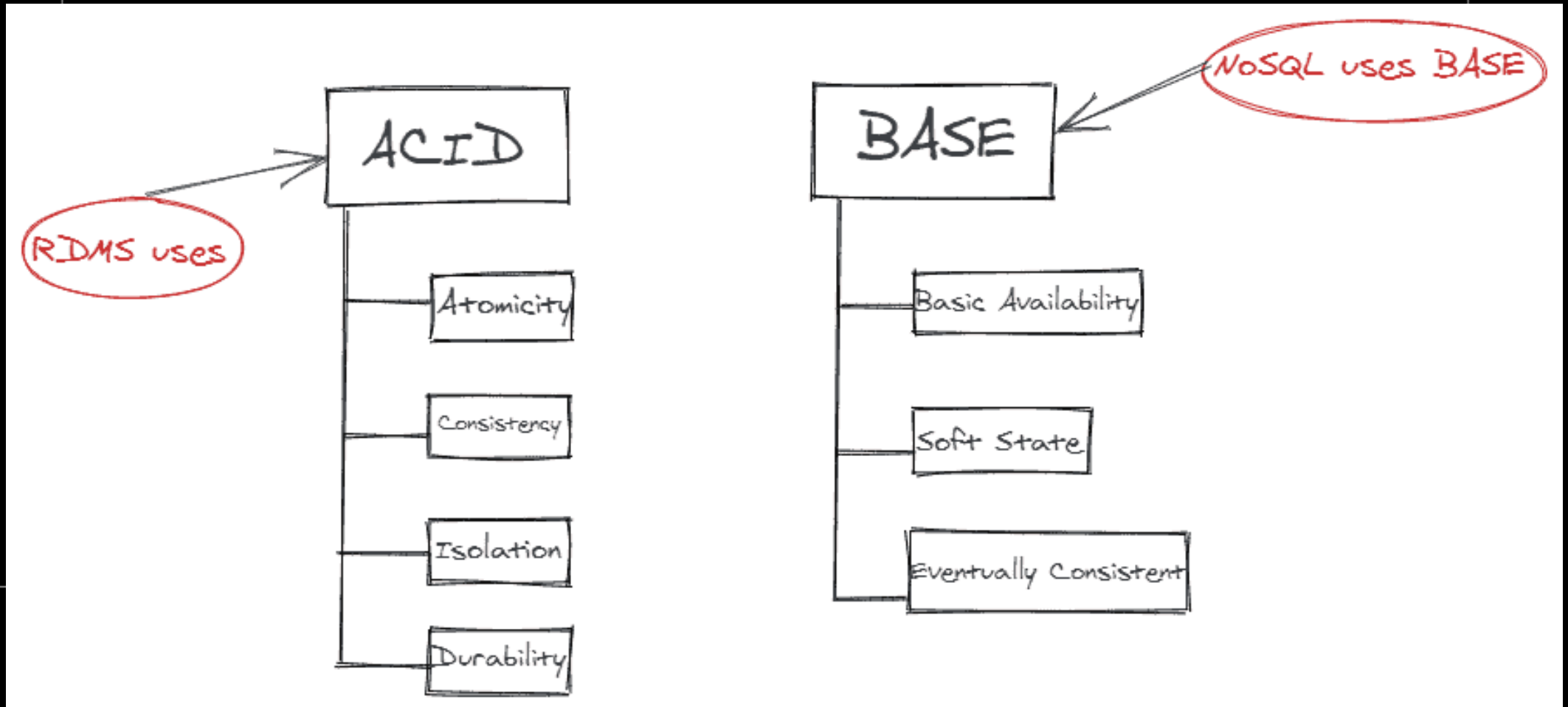
45697056

Relembrando...

Modelos de bancos de dados modernos:

- **Documento**: flexível, dados em JSON/BSON. Cada registro pode ter estrutura diferente. Forte em catálogos, perfis, conteúdo variável. Ex: MongoDB, CouchDB.
- **Chave-valor**: Estrutura mínima: uma chave, um valor opaco. Latência ultrabaixa, escala trivial. Dominante em cache e sessões. Ex: Redis, DynamoDB.
- **Colunar (Wide Column)**: Armazena dados por coluna, não por linha. Otimizado para agregações analíticas em grandes volumes. Ex: Cassandra, HBase.
- **Grafos**: Entidades e relacionamentos como cidadãos de primeira classe. Superior quando a conexão entre dados é tão importante quanto o dado em si. Ex: Neo4j. Casos de uso: detecção de fraude, redes sociais.
- **Vetorial**: Armazena embeddings, representações numéricas de texto, imagem e áudio. Consultas por similaridade semântica, não por igualdade exata. Vital para GenIA. Ex: Pinecone, Weaviate, pgvector.
- **Multimodelo**: SGBDs que suportam mais de um modelo simultaneamente no mesmo engine. Ex: ArangoDB, CosmosDB, FaunaDB.

Diferenças na operação



ACID vs BASE

A escolha entre consistência forte e disponibilidade se reflete nos modelos de consistência adotados por cada categoria de banco:

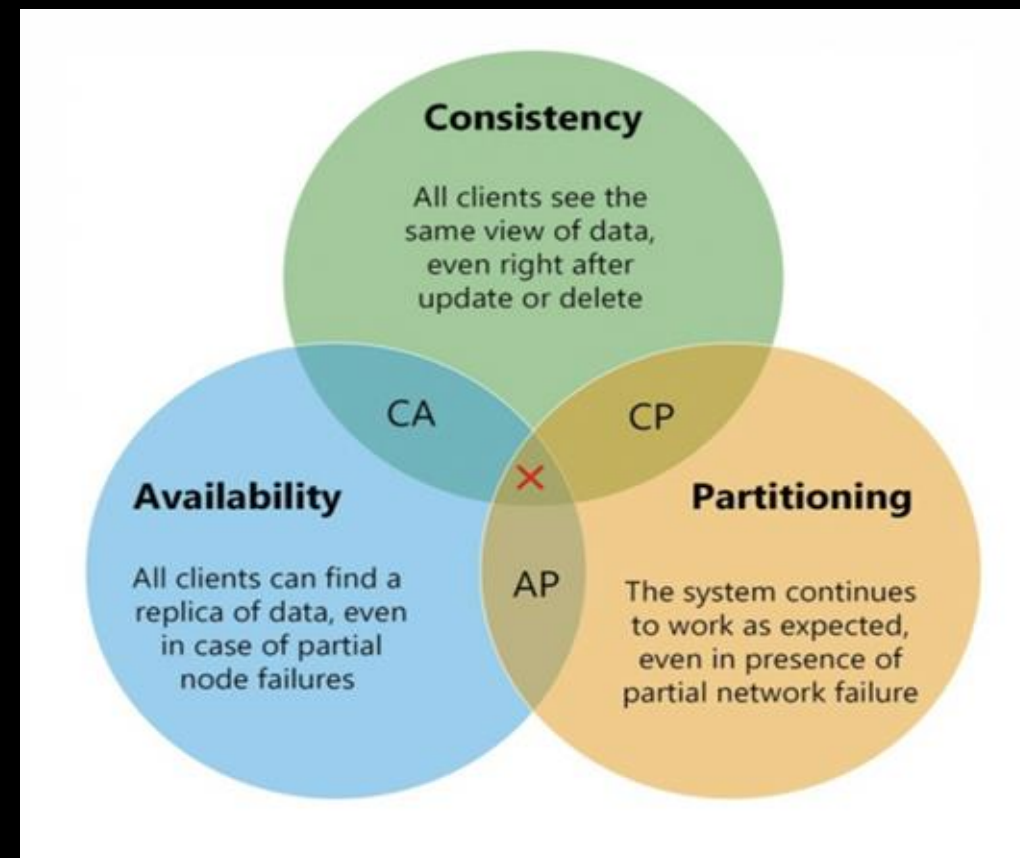
- **ACID — modelo dos bancos relacionais**
 - Garante que cada transação seja executada de forma completa e confiável
 - Consistência é imediata — após um commit, todos enxergam o novo valor
 - Custo: coordenação entre nós é cara em sistemas distribuídos
- **BASE — modelo predominante nos bancos NoSQL distribuídos**
 - Basically Available — o sistema responde sempre, mesmo durante falhas parciais
 - Soft State — o estado do sistema pode mudar ao longo do tempo mesmo sem novas escritas, devido à propagação de réplicas
 - Eventually Consistent — o sistema converge para consistência após um período, sem garantia de quando

A escolha entre ACID e BASE não é técnica — é de negócio. Um sistema financeiro não tolera inconsistência. Um feed de redes sociais tolera ver um post com alguns segundos de atraso.

O Teorema de CAP

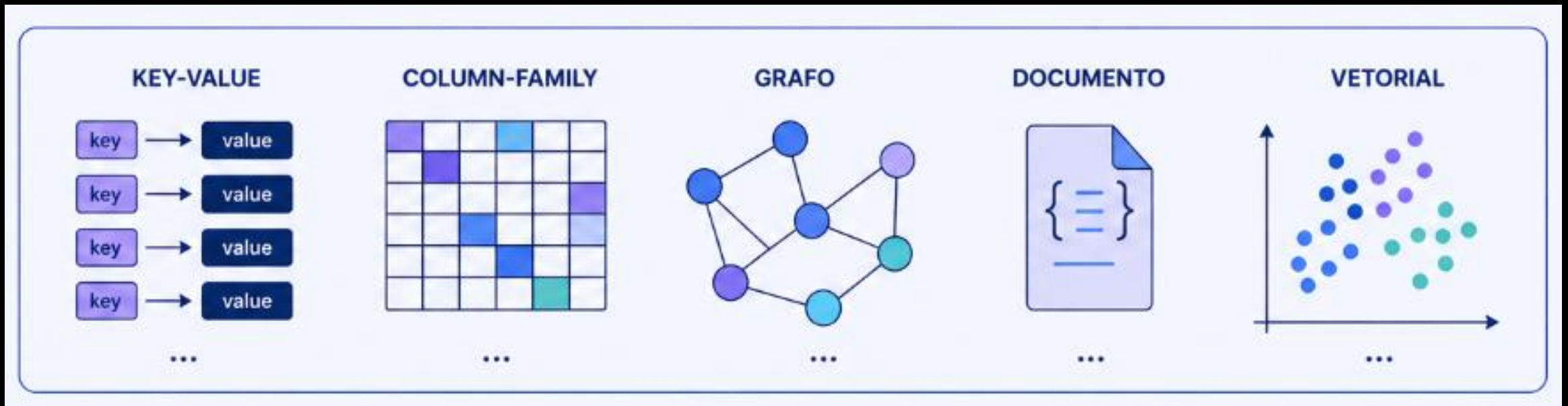
Em 2000, Eric Brewer propôs que sistemas distribuídos de dados não conseguem garantir simultaneamente mais de duas das três propriedades abaixo:

- Consistência (C) — todos os nós enxergam o mesmo dado ao mesmo tempo. Uma leitura sempre retorna o valor mais recente escrito
- Disponibilidade (A) — toda requisição recebe uma resposta, mesmo que não seja o dado mais recente
- Particionamento (P) — o sistema continua operando mesmo quando há falha de comunicação entre nós

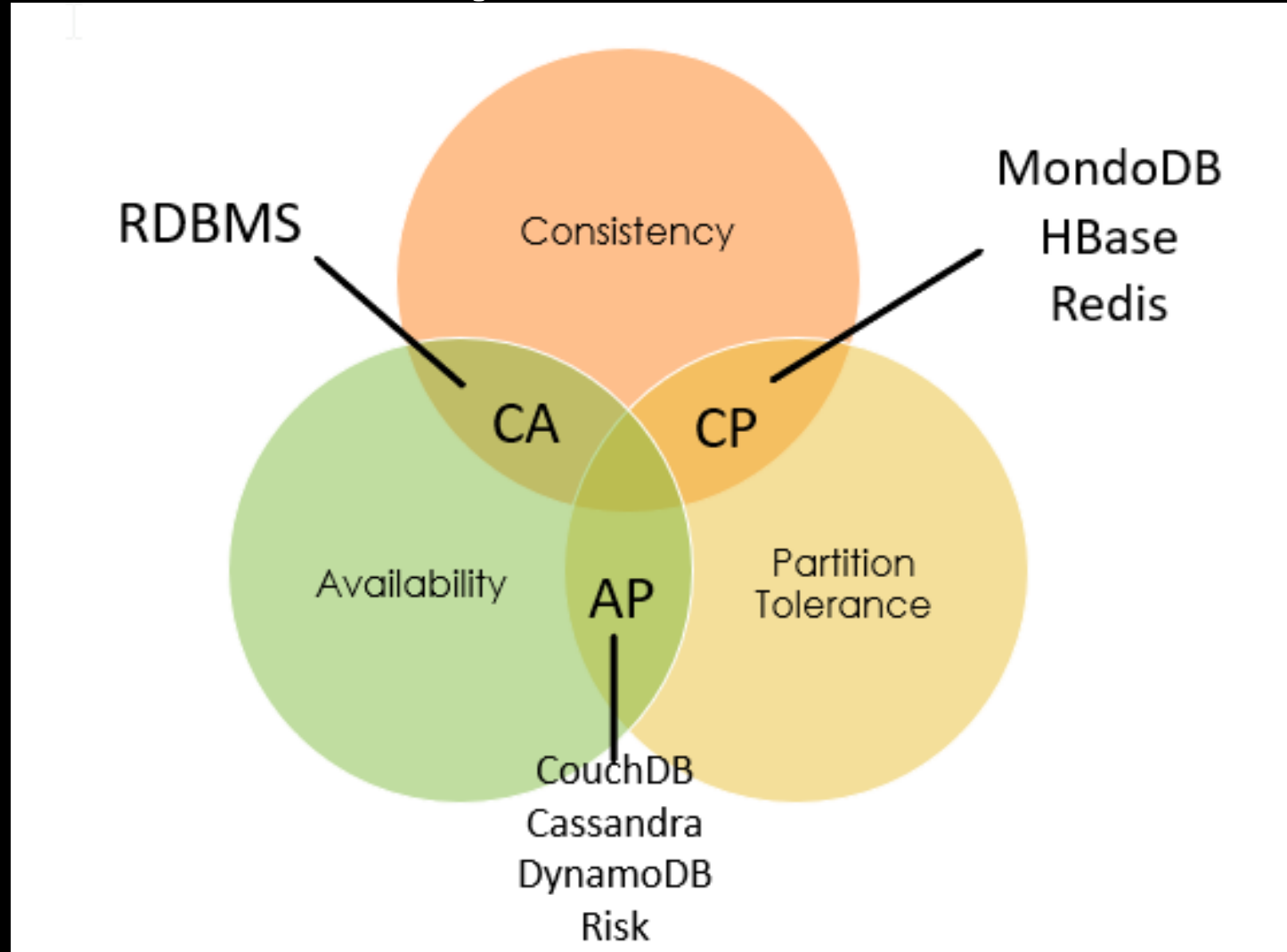


Na prática, falhas de rede em sistemas distribuídos são inevitáveis, de modo que P não é opcional. A escolha real é entre C e A: o sistema prioriza consistência (pode ficar indisponível durante uma partição) ou disponibilidade (pode retornar dados desatualizados). Bancos relacionais tradicionais operam no quadrante CA — assumem rede confiável e priorizam consistência e disponibilidade. Bancos NoSQL distribuídos geralmente escolhem CP ou AP.

Tipos de NoSQL



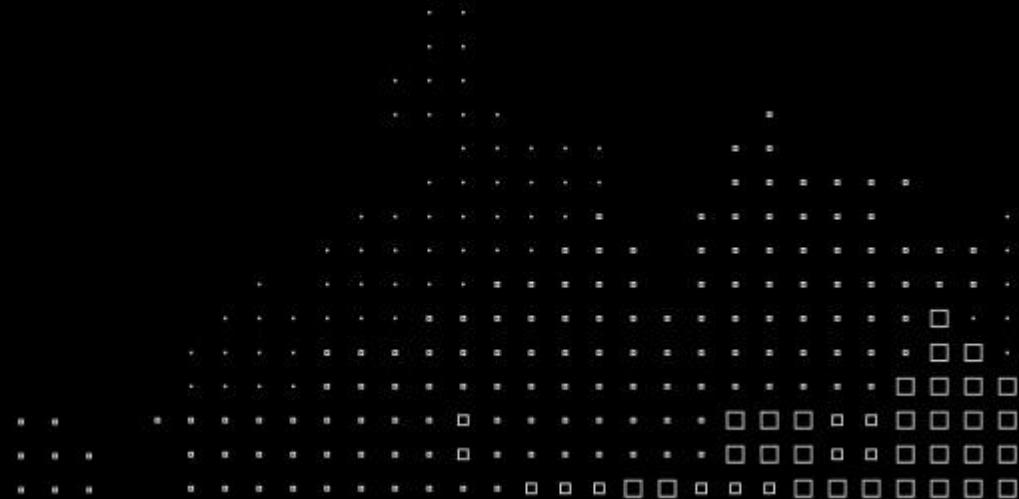
Alguns exemplos de NoSQL



Data Architecture, Integration and Ingestion

Capítulo 2: Bancos de dados NoSQL

Bancos de dados colunares



O modelo colunar — como funciona

No modelo relacional, os dados são armazenados linha a linha: todos os atributos de um registro ficam juntos no disco. Para recuperar uma coluna específica de milhões de registros, o banco precisa ler todas as linhas completas.

O modelo colunar inverte essa organização: os dados são armazenados coluna a coluna: todos os valores de um mesmo atributo ficam contíguos no disco. Implicações práticas:

- **Leitura**: consultas que acessam poucos atributos de muitos registros são drasticamente mais rápidas. O banco lê apenas as colunas necessárias, ignorando o restante
- **Escrita**: escritas pontuais em muitas colunas diferentes têm custo maior que no modelo linha. O modelo colunar é otimizado para leitura analítica e escrita em volume, não para atualizações frequentes de registros individuais
- **Compressão**: valores contíguos do mesmo tipo comprimem muito melhor. Colunas com baixa cardinalidade (ex: status, categoria) atingem taxas de compressão muito elevadas
- **Família de colunas**: bancos como o Cassandra estendem o conceito: colunas são agrupadas em famílias, permitindo que cada linha tenha um conjunto diferente de colunas dentro de uma mesma família. Isso combina flexibilidade de esquema com organização física eficiente.

Relacional vs Colunar — comparativo direto

	Relacional	Colunar
Organização física	Por linha	Por coluna / família de colunas
Esquema	Rígido, definido antes da inserção	Flexível por família de colunas
Consistência	ACID — forte e imediata	BASE — eventual por padrão
Escala	Vertical (scale-up)	Horizontal (scale-out)
Leitura analítica	Custo alto em grandes volumes	Otimizada — lê só as colunas necessárias
Escrita	Otimizada para registros individuais	Otimizada para alto volume
Joins	Suportados nativamente	Não suportados — modelagem orientada a query
Melhor para	Sistemas transacionais, integridade de negócio	IoT, logs, séries temporais, alta escrita

Referências: Chang, F. et al. (2006). *Bigtable: A Distributed Storage System for Structured Data*. OSDI.
Sadalage, P.J.; Fowler, M. (2012). *NoSQL Distilled*. Addison-Wesley.

Bancos colunares

☐ include secondary database models

13 systems in ranking, June 2026

Rank			DBMS	Database Model	Score		
Jun 2026	May 2026	Jun 2025			Jun 2026	May 2026	Jun 2025
1.	1.	1.	Apache Cassandra	Wide column, Multi-model 	102.97	+1.30	-5.30
2.	2.	↑ 3.	Microsoft Azure Cosmos DB	Multi-model 	21.65	-0.44	-0.78
3.	3.	↓ 2.	Apache HBase	Wide column	20.95	+1.18	-1.74
4.	4.	4.	ScyllaDB	Wide column, Multi-model 	4.73	+0.29	+0.77
5.	5.	5.	Datastax Enterprise	Wide column, Multi-model 	3.15	-0.07	-0.47
6.	6.	6.	Microsoft Azure Table Storage	Wide column	2.80	+0.01	+0.19
7.	7.	7.	Google Cloud Bigtable	Multi-model 	2.50	-0.02	-0.09
8.	8.	8.	Apache Accumulo	Wide column	2.44	+0.02	+0.27
9.	9.	9.	Amazon Keyspaces	Wide column	1.10	+0.00	+0.12
10.	10.	10.	HPE Ezmeral Data Fabric	Multi-model 	0.78	+0.02	-0.08
11.	11.	11.	Elassandra	Wide column, Multi-model 	0.38	0.00	+0.09
12.	12.	12.	Alibaba Cloud Table Store	Wide column	0.06	+0.01	-0.07
13.	13.	13.	SWC-DB	Wide column, Multi-model 	0.01	+0.00	0.00

Mais relevantes:

- ✓ Cassandra — projeto Apache, adotado em larga escala por Netflix, Apple, Instagram. Foco em alta disponibilidade e escrita distribuída
- ✓ Azure Cosmos DB — multimodelo da Microsoft, suporta API Cassandra
- ✓ HBase — roda sobre HDFS, integrado ao ecossistema Hadoop. Foco em consistência forte
- ✓ ScyllaDB — reimplementação do Cassandra em C++, com latências menores

Quando usar o modelo colunar

O modelo colunar não é uma substituição ao relacional — é uma escolha arquitetural para contextos específicos onde o relacional cede.

Use colunar quando:

- O volume de escrita é alto e contínuo — IoT, logs, métricas, eventos
- Os dados têm uma chave de identificação clara e um conteúdo variável por registro
- A leitura é analítica — poucos atributos, muitos registros
- A escala horizontal é requisito — distribuição geográfica, múltiplos datacenter
- A disponibilidade é crítica — o sistema não pode parar mesmo com falha de nós

Evite colunar quando:

- O modelo de dados é fortemente relacional — muitas entidades com joins complexos
- As operações exigem consistência forte e imediata — sistemas financeiros, transacionais
- Os dados são frequentemente atualizados em campos individuais
- O time não tem maturidade para modelagem orientada a query — o Cassandra exige que o modelo seja desenhado a partir das queries, não das entidades

Banco colunar: Cassandra

O Cassandra foi desenvolvido originalmente pelo Facebook em 2007 para resolver um problema específico: busca em caixa de entrada com alto volume de escrita e leitura distribuída globalmente. Em 2008 foi liberado como open source e em 2010 tornou-se projeto top-level da Apache Software Foundation.



Características centrais:

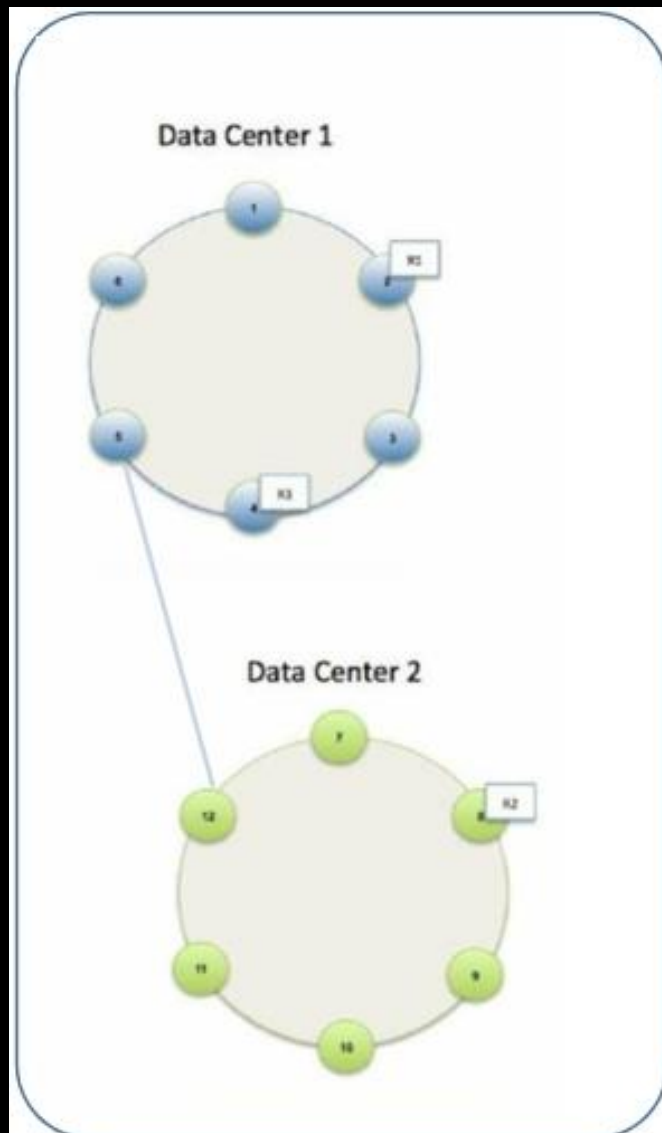
- Arquitetura peer-to-peer — sem nó mestre, sem ponto único de falha
- Escala horizontal linear — dobrar o número de nós dobra a capacidade de throughput
- Consistência configurável — o nível de consistência é definido por operação, não por banco
- Linguagem de consulta própria — CQL (Cassandra Query Language), sintaticamente similar ao SQL mas com restrições importantes

Adoção no mercado: Usado por Netflix, Apple, Instagram, Uber e Spotify para casos de uso de alta escrita e disponibilidade contínua — séries temporais, histórico de atividade, métricas em tempo real, mensageria.

Cassandra: características arquiteturais

- **Peer-to-peer e sem ponto único de falha:** Todos os nós têm o mesmo papel — não há nó mestre. A falha de um ou mais nós não interrompe o serviço. O cluster continua respondendo enquanto houver quórum disponível.
- **Escala horizontal linear:** A capacidade de throughput cresce proporcionalmente ao número de nós. Dobrar os nós dobra a capacidade de leitura e escrita — comportamento raro em sistemas distribuídos e impossível em bancos relacionais tradicionais.
- **Consistência configurável por operação:** O Cassandra não impõe um único nível de consistência. Cada operação define seu próprio nível — desde eventual (ONE) até forte (ALL ou QUORUM). Isso permite equilibrar consistência e disponibilidade conforme o requisito de cada caso de uso.
- **Gravações otimizadas:** Escritas são gravadas primeiro em memória (Memtable) e em log sequencial (CommitLog), depois persistidas em disco (SSTable). Esse mecanismo torna as gravações extremamente rápidas e é central para os casos de uso de alto volume.

Arquitetura Cassandra



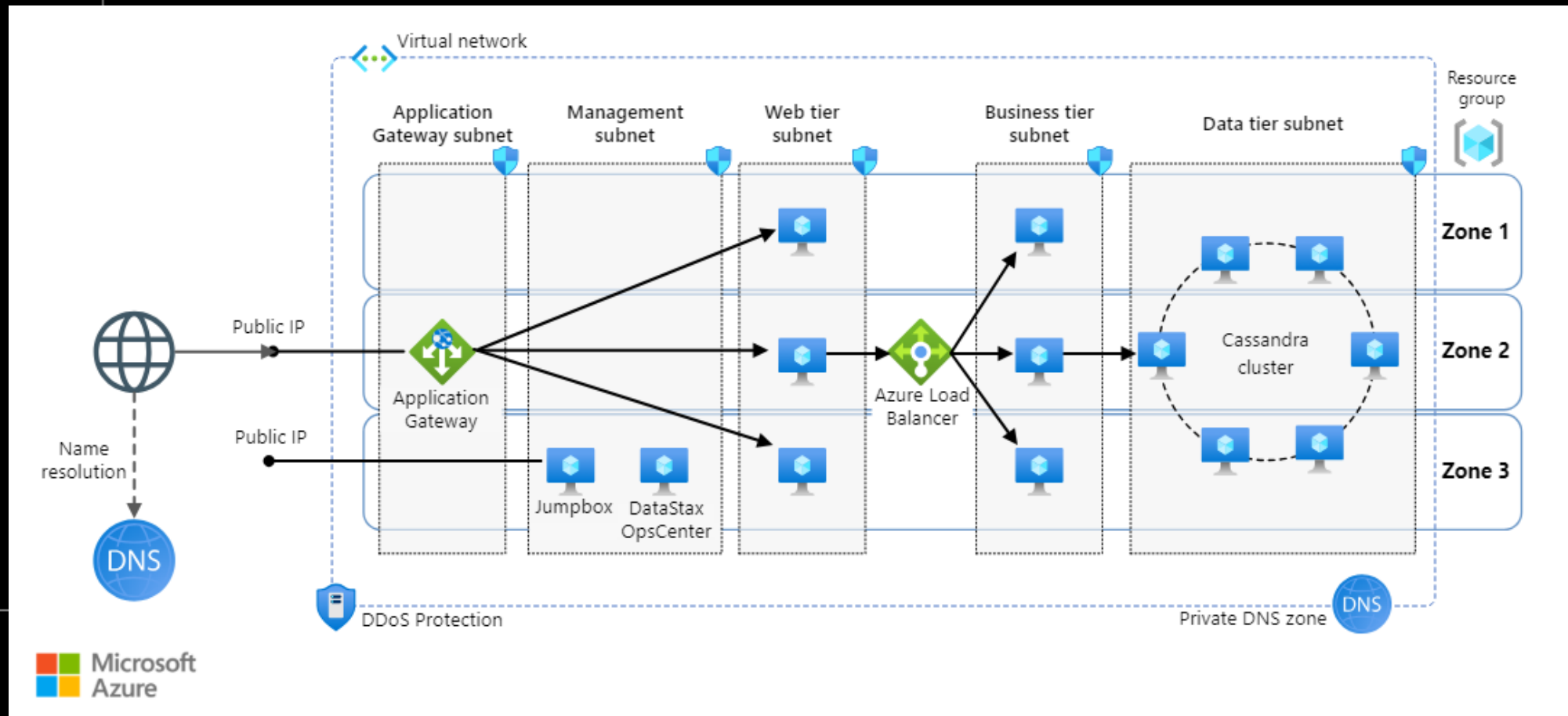
Componentes Cassandra:

- **Nó:** unidade básica de armazenamento. Cada nó é responsável por uma faixa de dados determinada pelo particionador. Todos os nós são equivalentes — não há distinção entre primário e secundário
- **Data Center:** agrupamento lógico de nós, geralmente correspondendo a uma região geográfica ou separação entre cargas de trabalho (ex: OLTP vs analytics)
- **Cluster:** conjunto de datacenters. Os dados são replicados entre datacenters conforme a estratégia de replicação configurada

Particionamento

Os dados são distribuídos entre os nós via consistent hashing — cada registro é direcionado ao nó responsável pelo intervalo que contém o hash de sua partition key. Isso garante distribuição uniforme e permite adicionar ou remover nós sem reorganizar todos os dados.

Arq. Cassandra em Azure (ex.)



Replicação de dados

Os dados são replicados para garantir nenhum ponto único de falha em caso de falha de hardware ou rede/conectividade, neste caso o Cassandra coloca réplicas de dados em diferentes nós com base nesses dois fatores:

Onde colocar a próxima réplica é determinado pela **Estratégia de Replicação**. Enquanto o número total de réplicas colocadas em nós diferentes é determinado pelo **Fator de Replicação**.

Um fator de replicação significa que há apenas uma única cópia de dados, enquanto três fatores de replicação significam que há três cópias dos dados em três nós diferentes. Para garantir que não haja um único ponto de falha, o fator de replicação deve ser três.

45697056

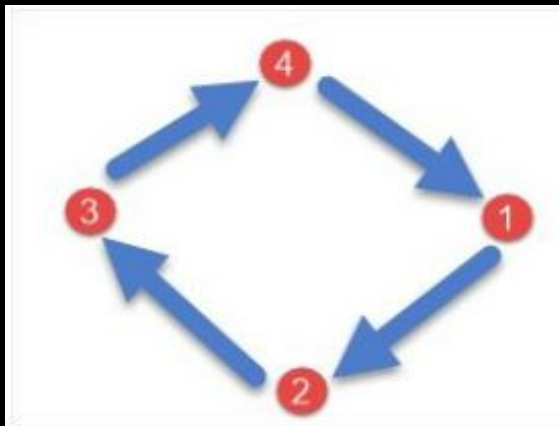
Estratégia de replicação

Existem dois tipos de estratégias de replicação no Cassandra:

SimpleStrategy

NetworkTopologyStrategy

O **SimpleStrategy** é usado quando você tem apenas um data center. SimpleStrategy coloca a primeira réplica no nó selecionado pelo particionador. Depois disso, as réplicas restantes são colocadas no sentido horário no anel do nó.

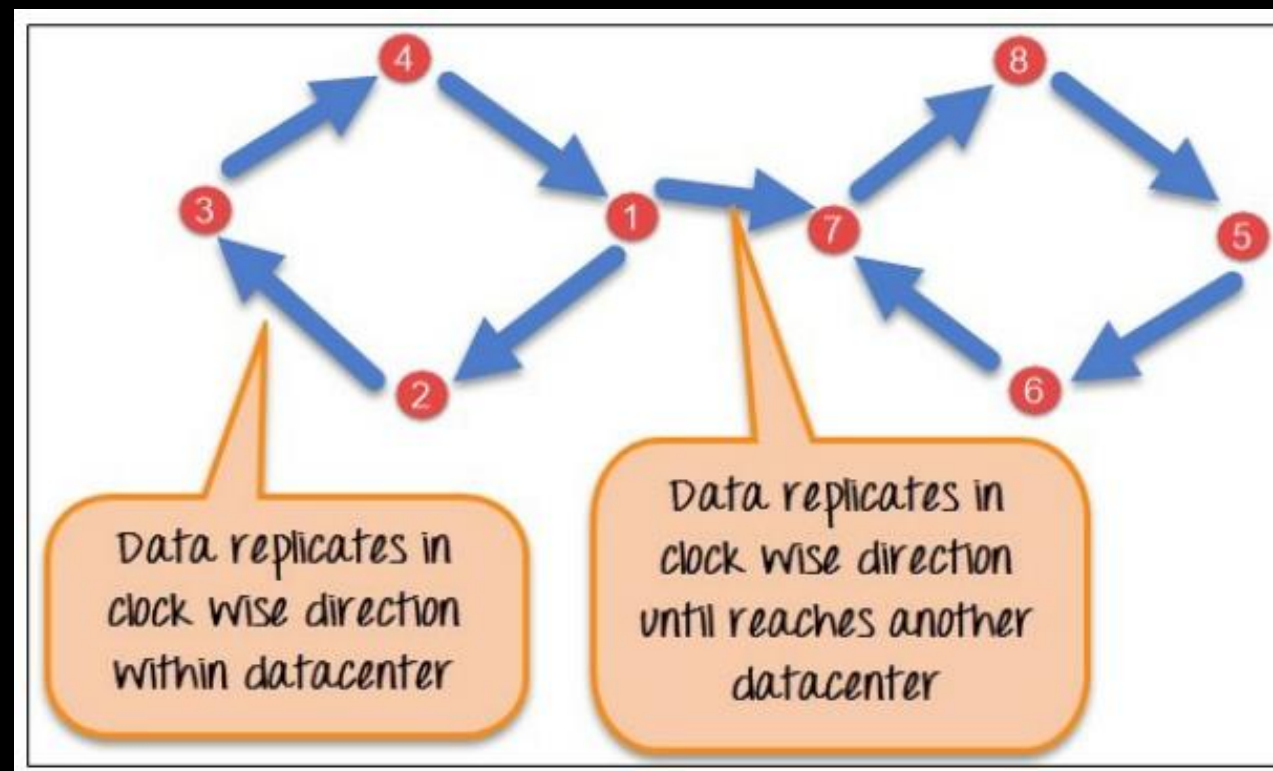


45697056

Estratégia de replicação

NetworkTopologyStrategy é usado quando você tem mais de dois data centers. As réplicas são definidas para cada data center separadamente. **NetworkTopologyStrategy** coloca réplicas no sentido horário no anel até atingir o primeiro nó em outro rack.

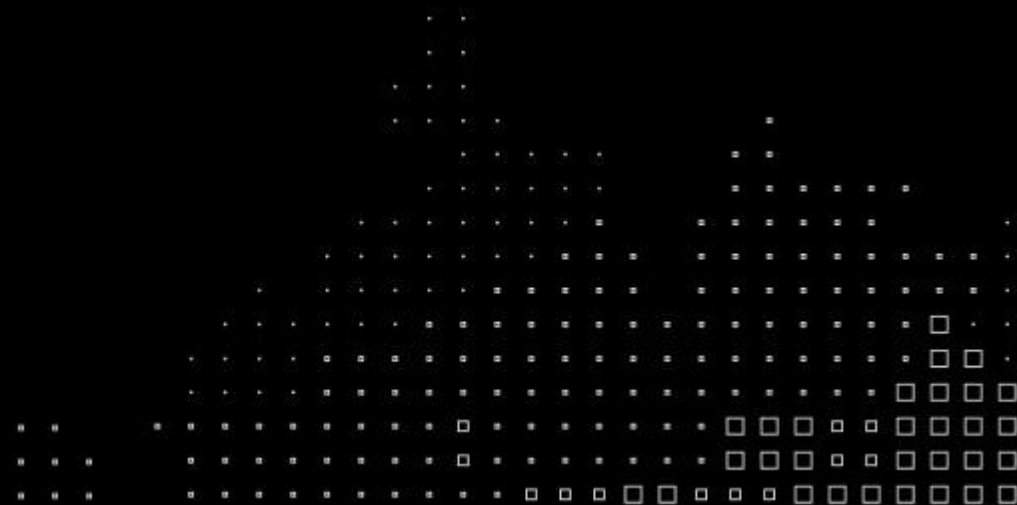
Essa estratégia tenta colocar réplicas em diferentes racks no mesmo data center. Isso se deve à razão de que, às vezes, falhas ou problemas podem ocorrer no rack. Em seguida, as réplicas em outros nós podem fornecer dados.



Data Architecture, Integration and Ingestion

Capítulo 2: Bancos de dados NoSQL

Prática com Cassandra



Opção A: DataStax AstraDB

Pré-requisito: conta gratuita em

<https://astra.datastax.com>

Setup:

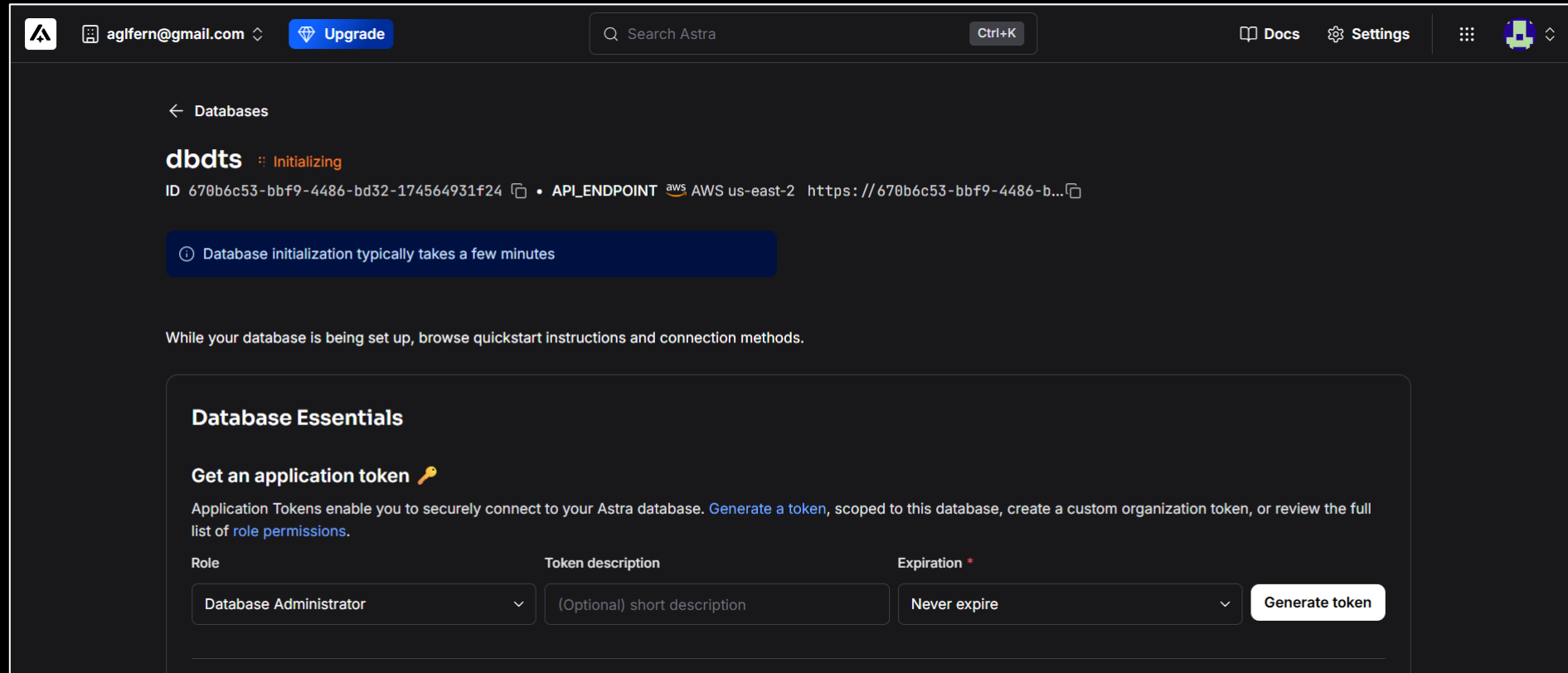
- Crie um banco com o nome **dbdts**, provider e região de sua preferência.
- Para keyspace name, use **ksdts**.
- Use o provider default: Amazon Web Services
- Use a região us-east-2 (que normalmente é a única disponível)

The screenshot shows the 'Create database' form in the DataStax AstraDB console. At the top, there are two tabs: 'Serverless (vector)' and 'Serverless (non-vector)'. The 'Serverless (non-vector)' tab is selected, indicated by a checkmark icon. Below the tabs, there are two input fields: 'Database name' with a placeholder 'db_name' and 'Keyspace name' with a placeholder 'keyspace_name'. Below these fields, there is a note: 'Give it a memorable name – this can't be changed later.' and 'Learn more about [keyspaces](#) and how to use them.' Below the note, there is a 'Provider' section with three buttons: 'Google Cloud', 'Amazon Web Services' (which is selected), and 'Microsoft Azure'. Below the provider section, there is a 'Region' section with a dropdown menu labeled 'Select a region'. At the bottom right of the form, there is a blue button labeled 'Create database'.

Opção A: DataStax AstraDB

Setup:

- Aguardar até que a criação tenha finalizado



Opção A: DataStax AstraDB

Setup:

- Em "Connect", gere um token de acesso e baixe o Secure Connect Bundle
- Acesse o CQL Console diretamente no browser — sem instalação

No CQL Console, crie o keyspace:

```
CREATE KEYSPACE IF NOT EXISTS ksdtb
WITH replication = {
    'class': 'SimpleStrategy',
    'replication_factor': 1
};

USE ksdtb;
```

Opção B: Docker + Cassandra

Pré-requisito: Docker instalado localmente

Criando a rede e subindo o container

```
# baixar imagem
docker pull Cassandra

# criar a rede
docker network create cassandra

# subir o container
docker run --rm -d --name cassandra \
  --hostname cassandra \
  --network cassandra \
  cassandra

# executar
docker exec -it cassandra cqlsh
```

Criando o keyspace

```
CREATE KEYSPACE IF NOT EXISTS ksdts
WITH replication = {
    'class': 'SimpleStrategy',
    'replication_factor': 1
};

USE ksdts;
```

Exercício Cassandra



Será disponibilizado um arquivo TXT com os scripts para executar os comandos a seguir. Copie e execute os comandos do arquivo e observe os resultados.

1. Criar um Container a partir da imagem (feito)
2. Verifique o estado do cluster Cassandra (feito)
3. Acesse o CLI do Cassandra **cqlsh** (feito)
4. Crie um keyspace chamado **ksdts** (feito)
5. Acesse o keyspace **ksdts**
6. Crie a tabela **aula_cassandra**
7. Insira um registro na tabela
8. Consulte a tabela (*)
9. Inclua um novo campo
10. Consulte a tabela (*)

Observaram algo de diferente na consulta?

Observaram algo de diferente na consulta?

45697056

Continuação exercício Cassandra...

Tipos de dados multivalorados

11. Criar a tabela `aula_cassandra_set`
12. Inserir um registro na tabela `aula_cassandra_set` (*)
13. Consultar o registro inserido
14. Criar a tabela `aula_cassandra_list`
15. Inserir um registro na tabela `aula_cassandra_list` (*)
16. Consultar o registro inserido
17. Criar a tabela `aula_cassandra_map`
18. Inserir um registro na tabela `aula_cassandra_map` (*)
19. Consultar o registro inserido
20. Criar a tabela `aula_cassandra_tuple` +
21. Inserir um registro na tabela `aula_cassandra_tuple` (*)
22. Consultar o registro inserido

Conjunto de coleções com um ou mais elementos ordenados

Conjunto com lista de elementos mantidos na ordem inserida

Conjunto com mapa de chave-valor

Conjunto capaz de armazenar conjuntos de dados. Aceita até 32768 campos

Continuação exercício Cassandra...

Índices e filtros

23. Inserir mais registros na tabela *aula_cassandra*
24. Consultar campo_texto na tabela *aula_cassandra*
25. Fazer nova consulta com *allow filtering* (*)
26. Criar um índice na tabela *aula_cassandra* (*)
27. Consultar o registro inserido usando índice
28. Inserir registros na tabela *aula_cassandra_map*
29. Criar índice map key
30. Consultar pela *chave* do campo map
31. Criar índice map value
32. Consultar pelo *valor* do campo map
33. Criar índice map entries
34. Consultar pela *entrada* do campo map

Em tabelas muito grandes, o tempo de consulta será elevado

A criação de índices é eficiente, mas pode aumentar os tempos de gravação

Continuação exercício Cassandra...

Exclusão de dados e tabelas

- 35. Exclua registros na tabela *aula_cassandra*
- 36. Excluir dados de uma coluna na tabela *aula_cassandra*
- 37. Excluir conteúdo campo set na *aula_cassandra_set*
- 38. Excluir conteúdo campo list na *aula_cassandra_list* (*)
- 39. Excluir conteúdo campo map na *aula_cassandra_map*
- 40. Faça um truncate na tabela *aula_cassandra_list*
- 41. Excluir um dos índices da tabela *aula_cassandra_map*
- 42. Excluir a tabela *aula_cassandra_set*
- 43. Criar um keyspace e exclui-lo para testar

O índice na lista inicia com 0

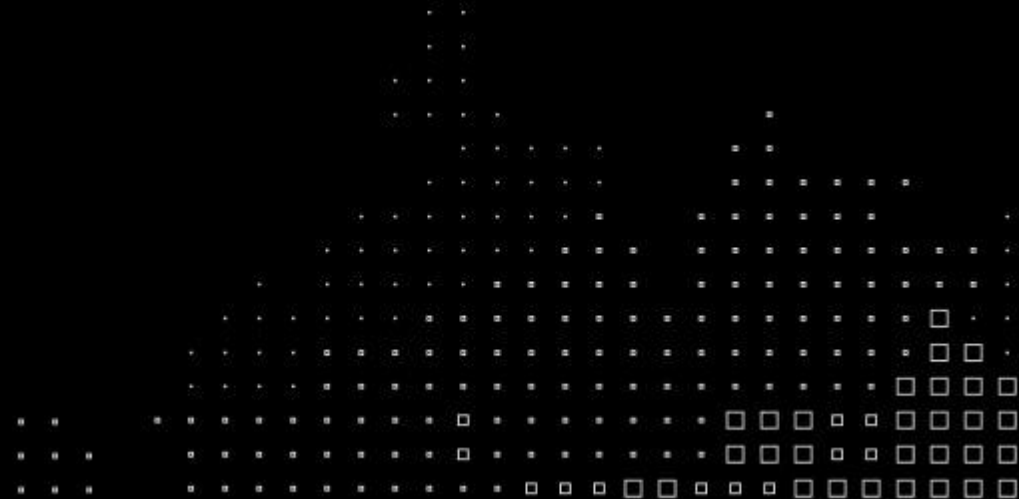
Desafios

1. O que acontece se inserir o mesmo valor duas vezes?
2. O que acontece se inserir a mesma chave, mas com valores diferentes?
3. Atualize registros nas tabelas
4. É possível fazer delete sem usar a primary key?
5. Faça updates nos campos multivalorados set, list e map
6. Inserir registros no campo set fora de ordem
7. Criar tabela com campos tuple aninhados

Data Architecture, Integration and Ingestion

Material adicional

Prática adicional
com **mySQL**



Exercício MySQL

Adicionalmente, usando seu Docker local e o container SQL criado anteriormente, você pode executar os seguintes comandos para exercitar o uso de uma base relacional. Novamente, o script será disponibilizado com os comandos.

1. Criar um Container a partir da imagem
2. Fazer a instalação do MySQL
3. Verificar o estado do serviço e iniciar o serviço MySQL
4. Acessar o CLI do Mysql
5. Criar um database chamado *db_dts*
6. Acessar o database criado
7. Criar uma uma tabela chamada *aula_mysql*
8. Inserir um registro na tabela criada
9. Consultar os registros da tabela *aula_mysql*

45697056

Continuação exercício MySQL...

Usuários e acessos

10. Criar um usuário chamado *fiap* com a senha *f1@p*
11. Sair do CLI do MySQL
12. Acessar o CLI MySQL com o usuário *fiap* (*)
13. Inserir um registro na tabela *aula_mysql* (*).
14. Sair do CLI e reconectar com o usuário anterior
15. Dar permissão ao usuário *fiap* na tabela *aula_mysql*
16. Sair do CLI e reconectar com o usuário *fiap*
17. Inserir 1 registro na tabela
18. Consultar os registros da tabela *aula_mysql*

Com qual usuário estava conectado antes?

O que houve neste passo e porquê?

45697056

Continuação exercício MySQL...

Manipulação de dados

- 19. Alterar um registro na tabela *aula_mysql*
- 20. Consultar os registros da tabela *aula_mysql*
- 21. Excluir 1 registro da tabela *aula_mysql*
- 22. Consultar os registros da tabela *aula_mysql* (*)

Porque foi possível alterar e excluir registros?

Armazenamento dos dados

- 23. Sair do CLI do MySQL
- ~~24. Navegar até o diretório do database *db_dts*~~
- 25. Abrir o arquivo de dados da tabela *aula_mysql* (*)

É possível interpretar os dados do arquivo?

Continuação exercício MySQL...

Remoção dos acessos e dados



- 26. Abrir o CLI do MySQL com o usuário root
- 27. Revogue as permissões do usuário *fiap*
- 28. Remova a tabela *aula_mysql*
- 29. Remova o database *db_dts*

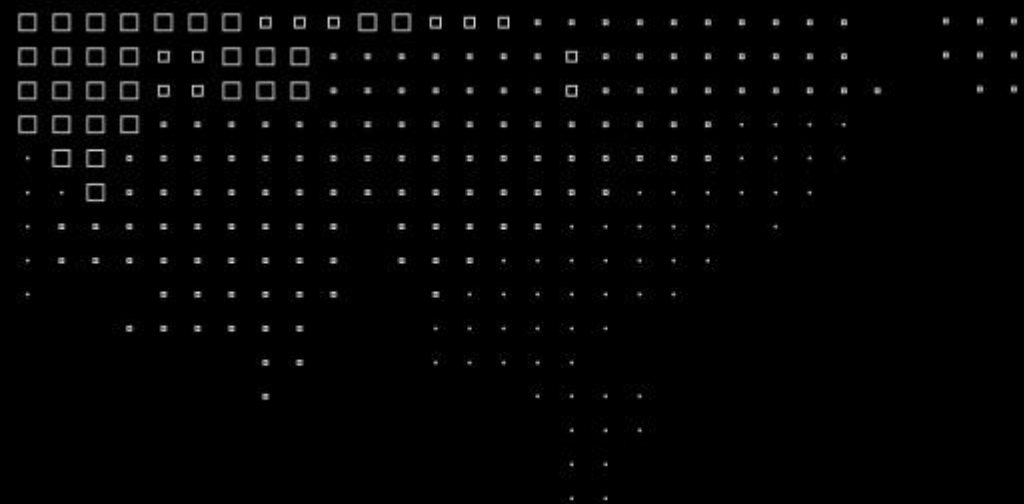
Armazenamento dos dados

- 30. Sair do CLI do MySQL
- ~~31. Navegar até o diretório do database *db_dts*^(*)~~

O que houve?

Desafios

1. Crie um database e uma tabela de clientes (id, nome, idade, e-mail)
2. Insira alguns registros nessa tabela
3. Explore o comando select mudando filtros e campos selecionados
4. Explore o comando update, atualizando alguns registros
5. Atualize mais de um campo por execução, múltiplos registros
6. Use o comando truncate table
7. Pare o serviço do mysql e inicie novamente


MBA⁺

Copyright © **2025-26** Prof. Ivan Gancev (com contribuições por Prof. Leandro Mendes)

Todos direitos reservados. Reprodução ou divulgação total ou parcial deste documento é expressamente proibido sem o consentimento formal, por escrito, do Professor Ivan Gancev
profivan.gancev@fiap.com.br

